

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ПОВОЛЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ПГТУ»)

Факультет (институт) ИиВТ

Направление подготовки 09.03.04

Программная инженерия

(наименование)

Выпускник: Фамилия Ибатуллин
Имя Рамир
Отчество Марселович

Тема ВКР «Разработка веб-приложения векторного редактора диаграмм и схем»

Кафедра информатики и системного программирования

Заведующий кафедрой Бородин А.В., к.э.н., профессор

(ф.и.о. уч. степень, звание, должность, подпись)

Руководитель Баев А.А., доцент ИиСП, к.т.н

(ф.и.о. уч. степень, звание, должность, подпись)

Консультанты _____

(ф.и.о. уч. степень, звание, должность, подпись)

№ приказа об утверждении темы ВКР

212-ОН

ВКР начата

10.02.26

ВКР закончена

10.06.26

№ приказа о допуске к защите ВКР

1056-ОН

Оценка Государственной экзаменационной комиссии по защите _____

Декан факультета

(Директор института)

(Кречетов А.А.)

Секретарь Государственной
экзаменационной комиссии

(Глушкова О.В.)

« ____ » _____ 2026 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ПОВОЛЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ПГТУ»)

УТВЕРЖДАЮ

Зав. кафедрой ИиСП _____
_____ (А.В. Бородин)

« ____ » _____ 2026 г.

**ЗАДАНИЕ
НА ВЫПОЛНЕНИЕ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ**

Выпускник Ибатуллин Рамир Марселович
Направление подготовки (специальность) 09.03.04 Программная инженерия
Форма обучения очная Группа ПС- 42
Тема выпускной квалификационной работы
«Разработка веб-приложения векторного редактора диаграмм и схем»

Содержание задания

1. Исходные данные В рамках выпускной квалификационной работы необходимо разработать программное средство Vision Diagram — интеллектуальный веб-редактор диаграмм и таблиц. Система должна обеспечивать создание и редактирование схем, работу с таблицами и списками, построение связей между объектами, а также генерацию и изменение диаграмм по текстовому запросу. Приложение должно поддерживать импорт, экспорт, локальное сохранение данных и переключение темы интерфейса
2. Содержание пояснительной записки с указанием разделов ВКР введение, постановка задачи, аналитическая часть, конструкторская часть, техническая документация, заключение
3. Содержание графической части презентация, состоящая из 7 слайдов в формате 16:9
(перечень графического материала, число листов с указанием формата)

КОНСУЛЬТАНТЫ:

(должность, Ф.И.О., подпись, дата)

(должность, Ф.И.О., подпись, дата)

(должность, Ф.И.О., подпись, дата)

(должность, Ф.И.О., подпись, дата)

(должность, Ф.И.О., подпись, дата)

РУКОВОДИТЕЛЬ ВКР

доцент ИиСП, к.т.н., Баев А.А.

(должность, Ф.И.О., подпись, дата)

Задание принял к исполнению Ибатуллин Рамир Марселович

(Ф.И.О. студента)

10.02.26

(дата и подпись)

Задание зарегистрировано:

ЗАВЕДУЮЩИЙ КАФЕДРОЙ Бородин А.В.

(подпись)

«_____» _____ 2026 г.

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке программного средства Vision Diagram, предназначенного для создания, редактирования и анализа диаграмм и таблиц в веб- среде. Актуальность темы обусловлена необходимостью ускорения подготовки визуальных моделей, используемых при проектировании информационных систем, описании алгоритмов, моделировании бизнес- процессов и подготовке технической документации. Традиционные редакторы диаграмм предоставляют широкий набор инструментов, однако требуют значительного объема ручных действий со стороны пользователя [1]. Использование методов искусственного интеллекта позволяет сократить время построения схем и упростить работу с графическими объектами.

В работе рассматривается проектирование и реализация интеллектуального редактора, включающего клиентскую и серверную части. Клиентская часть реализована как одностраничное веб- приложение на языке TypeScript с использованием библиотеки React. Серверная часть разработана на платформе Node.js с использованием фреймворка Express и обеспечивает обработку запросов к языковой модели через Ollama. В качестве внутреннего представления диаграммы используются типизированные структуры данных, содержащие описание геометрии фигур, их текстового содержимого и связей между объектами.

В первой главе рассматриваются постановка задачи, проблематика предметной области, актуальность разработки и требования к создаваемому программному средству. Во второй главе выполняется анализ аналогичных решений, обосновывается выбор архитектуры и стека технологий. В третьей главе приводится описание структуры приложения, модели данных, алгоритмов работы редактора и принципов интеграции модуля искусственного интеллекта. Четвертая глава содержит элементы технической документации и руководство пользователя по запуску и эксплуатации системы.

Практическим результатом работы является веб- приложение Vision Diagram, позволяющее создавать фигуры нескольких типов, формировать связи между ними, редактировать содержимое таблиц и списков, выполнять импорт и экспорт диаграмм в формате draw.io, а также генерировать и изменять схему по текстовому описанию пользователя.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	8
ВВЕДЕНИЕ	9
1. ПОСТАНОВКА ЗАДАЧИ	12
1.1 Описание проблемы	12
1.2 Актуальность	12
1.3 Постановка задачи	13
1.4 Требования к эксплуатационным характеристикам	14
1.5 Требования к качеству исходного кода	15
2. АНАЛИТИЧЕСКАЯ ЧАСТЬ	17
2.1 Анализ подобных систем	17
2.1.1 Критерии сравнения существующих решений	18
2.1.2 Сравнение с diagrams.net	18
2.1.3 Сравнение с Lucidchart и Miro	19
2.1.4 Сравнение с Microsoft Visio и настольными редакторами	19
2.1.5 Итоги сравнительного анализа	19
2.2 Анализ разрабатываемого решения	20
2.3 Выбор архитектуры	20
2.3.1 Выбор архитектуры приложения	20
2.3.2 Выбор архитектуры клиентской части	21
2.3.3 Выбор архитектуры серверной части	21
2.4 Выбор технологий	21
2.4.1 Выбор языка программирования	21
2.4.2 Выбор технологий клиентской части	22
2.4.3 Выбор технологий серверной части	22
2.4.4 Выбор платформы для работы с языковой моделью	22
2.4.5 Выбор инструментов разработки	23
2.4.6 Обоснование выбора SVG как основы графического редактора	23
2.4.7 Обоснование выбора Zustand для хранения состояния	23
2.4.8 Причины выбора локальной языковой модели	24
3. КОНСТРУКТОРСКАЯ ЧАСТЬ	25
3.1 Структура приложения	25

3.1.1 Структура клиентской части	25
3.1.2 Структура серверной части	26
3.1.3 Взаимодействие клиентской и серверной частей	27
3.2 Модель данных диаграммы	27
3.2.1 Структура данных фигуры	29
3.2.2 Структура данных связи	29
3.2.3 Структура действия интеллектуального модуля	29
3.3 Реализация графического редактора	30
3.4 Реализация интеллектуального модуля	32
3.5 Импорт и экспорт диаграмм	34
3.6 Пользовательские сценарии работы	36
3.7 Тестирование программного средства	37
3.7.1 Цели и задачи тестирования	37
3.7.2 Проверка клиентской части	38
3.7.3 Проверка серверной части	38
3.7.4 Проверка интеллектуального модуля	38
3.7.5 Проверка импорта и экспорта	39
3.7.6 Результаты тестирования и выводы	39
3.7.7 Контрольные примеры запросов к интеллектуальному модулю	40
4. ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ	41
4.1 Запуск системы	41
4.2 Работа с редактором	41
4.2.1 Общая компоновка пользовательского окна	43
4.2.2 Верхняя панель инструментов и назначение кнопок	44
4.2.3 Рабочая область и поведение холста	46
4.2.4 Выделение, перемещение, изменение размеров и поворот объектов	46
4.2.5 Редактирование простых фигур	47
4.2.6 Работа со списком как отдельным типом объекта	48
4.2.7 Работа с таблицей и встроенные управляющие элементы	49
4.2.8 Создание связей и настройка типа стрелки	49
4.2.9 Строка состояния как средство обратной связи	50
4.3 Работа с интеллектуальным помощником	51

4.3.1 Структура и элементы панели интеллектуального помощника	52
4.3.2 Логика взаимодействия с AI и типы пользовательских запросов	52
4.3.3 Практическая роль AI- панели в пользовательском процессе	58
4.4 Импорт и экспорт файлов	53
4.4.1 Поведение интерфейса при импорте	54
4.4.2 Поведение интерфейса при экспорте	54
4.5 Типовые ошибки и способы их устранения	55
4.6 Ограничения текущей версии и рекомендации пользователю	55
ЗАКЛЮЧЕНИЕ	57
Перспективы дальнейших исследований	57
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	59
ПРИЛОЖЕНИЕ А. ОПИСАНИЕ ПРОГРАММЫ	60
ПРИЛОЖЕНИЕ Б. РУКОВОДСТВО ОПЕРАТОРА	62
ПРИЛОЖЕНИЕ В. ПРОТОКОЛ ФУНКЦИОНАЛЬНОГО ТЕСТИРОВАНИЯ	64
ПРИЛОЖЕНИЕ Г. СПЕЦИФИКАЦИЯ СТРУКТУР ДАННЫХ И ОБМЕНА	66

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей работе используются следующие определения, обозначения и сокращения.

- **AI** - искусственный интеллект, применяемый в работе как общий термин для обозначения интеллектуального модуля, генерирующего действия над диаграммой.
- **API** - программный интерфейс взаимодействия между клиентской и серверной частями приложения.
- **JSON** - текстовый формат обмена структурированными данными, используемый для передачи состояния диаграммы и ответов интеллектуального модуля.
- **Node.js** - программная платформа для выполнения серверной части приложения [6].
- **Ollama** - локальная среда запуска и обращения к языковой модели, используемая сервером Vision Diagram [9].
- **REST** - архитектурный стиль взаимодействия по HTTP, примененный при построении серверных маршрутов.
- **SVG** - векторный формат графики, используемый как основа рабочей области редактора [10].
- **TypeScript** - типизированный язык программирования, применяемый при разработке клиентской и серверной частей [4].
- **Vite** - инструмент сборки и разработки клиентского приложения [5].
- **Zustand** - библиотека централизованного хранения состояния интерфейса [8].
- **draw.io** - формат и экосистема обмена диаграммами, с которыми обеспечивается совместимость приложения [1], [2].

ВВЕДЕНИЕ

В ходе дипломного проекта рассматривалась прикладная задача, с которой регулярно сталкиваются студенты и разработчики: нужно быстро превратить описание процесса, алгоритма или структуры программы в понятную визуальную схему. На практике такая схема может включать не только блоки и стрелки, но и таблицы, списки, поясняющие подписи и вспомогательные элементы. Диаграммы помогают быстрее увидеть связи между частями системы, согласовать терминологию и обнаружить логические разрывы еще до того, как описание переносится в код или техническую документацию [1], [10].

При работе с существующими редакторами заметна одна повторяющаяся проблема: большая часть времени уходит не на продумывание модели, а на механические операции. Пользователь выбирает фигуру, размещает ее на холсте, подгоняет размеры, вводит подпись, соединяет элементы стрелками и затем повторяет эти действия много раз. Если исходное описание уже существует в виде текста, такой перенос становится особенно однообразным: смысл процесса понятен, но его все равно приходится вручную собирать из отдельных графических объектов.

Для разрабатываемой системы важен и другой практический момент. В учебных работах и небольших инженерных проектах схема редко существует сама по себе: рядом с ней часто нужны таблица параметров, список этапов, перечень ролей или набор коротких пояснений. Поэтому редактор должен позволять работать с разными видами структурированных объектов в одной рабочей области, а не заставлять пользователя переходить между несколькими инструментами.

Идея Vision Diagram состоит в том, чтобы совместить обычное ручное редактирование с управлением через текстовый запрос. Пользователь может сначала получить черновую диаграмму по описанию, а затем уже привычными средствами переместить элементы, изменить подписи, соединить фигуры или уточнить таблицу. Такой подход не заменяет мышь и клавиатуру, а снимает наиболее рутинную часть начального построения схемы.

Объектом исследования является процесс создания и редактирования схем и таблиц в браузерном приложении. Предметом исследования являются программные методы построения редактора диаграмм, в котором ручные операции на SVG-холсте сочетаются с интеллектуальным модулем, возвращающим формализованные действия над объектами диаграммы.

Цель выпускной квалификационной работы заключается в разработке программного средства Vision Diagram, которое позволяет создавать и изменять диаграммы в браузере,

импортировать и экспортировать файлы формата draw.io, переключать оформление интерфейса и использовать языковую модель для генерации или уточнения структуры схемы.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать предметную область и существующие программные решения;
- определить требования к интеллектуальному редактору таблиц и схем;
- выбрать архитектуру приложения и обосновать используемый стек технологий;
- разработать клиентскую часть графического редактора;
- реализовать серверную часть для взаимодействия с языковой моделью;
- спроектировать модель данных, описывающую фигуры и связи диаграммы;
- реализовать функции редактирования, масштабирования, соединения элементов, импорта и экспорта;
- реализовать интеллектуальный модуль для генерации и изменения диаграмм по текстовому запросу;
- подготовить техническую документацию по эксплуатации системы.

Практическая значимость проекта состоит в том, что полученное приложение можно использовать как инструмент быстрого чернового моделирования. Оно подходит для учебных заданий, подготовки пояснительных схем к программным проектам, демонстрации алгоритмов и первичного описания архитектуры. Пользователь получает не статическую картинку, а редактируемую модель из фигур и связей, которую можно доработать вручную или передать во внешний редактор через формат draw.io.

При выполнении работы применялись анализ предметной области, сравнение существующих редакторов, проектирование клиент-серверного взаимодействия, типизация модели данных, разбиение интерфейса на компоненты и практическая проверка сценариев работы. Отдельное внимание уделялось тому, чтобы результат языковой модели попадал в редактор не как свободный текст, а как набор допустимых операций над диаграммой.

Новизна работы для данного дипломного проекта проявляется в способе объединения графического редактора и языковой модели. В Vision Diagram модель не рисует схему напрямую и не управляет интерфейсом. Она возвращает структурированный JSON-ответ с действием replace, add, update, remove или analyze, после чего клиентская часть сама применяет эти изменения к состоянию фигур и связей.

В образовательном контексте такой режим особенно полезен. Студент может начать с короткого описания алгоритма или архитектуры, получить первичную заготовку, а затем вручную

довести ее до нужного вида. В результате внимание переносится с однотипного перетаскивания блоков на проверку логики схемы, формулировку подписей и понимание структуры моделируемого процесса.

Структура пояснительной записки выстроена по этапам выполнения проекта. Сначала формулируются проблема и требования к системе, затем рассматриваются аналоги и обосновывается выбранный стек технологий. После этого описываются архитектура, модель данных, реализация редактора, работа AI-модуля, тестирование и пользовательская документация. В заключении приведены результаты и направления дальнейшего развития.

1. ПОСТАНОВКА ЗАДАЧИ

1.1 Описание проблемы

Исходная проблема проекта связана с тем, что визуальное описание информации обычно требует большого количества мелких ручных действий. Пользователю нужно не просто нарисовать прямоугольник или стрелку, а удерживать в голове общую структуру процесса, расположение блоков, связи между ними и сопроводительные данные. Когда схема дополняется таблицами или списками, число операций возрастает еще сильнее.

Традиционные редакторы хорошо решают задачу точного ручного построения, но почти не сокращают путь от текстовой постановки к первой версии диаграммы. Если пользователь уже написал последовательность действий алгоритма, состав модулей или этапы бизнес-процесса, ему все равно приходится вручную переносить это описание на холст. В дипломном проекте именно этот участок работы выбран как основной объект автоматизации.

Дополнительная сложность появляется при последующих правках. Небольшое изменение в тексте задачи может означать добавление нового блока, удаление связи, изменение направления стрелки или перестройку нескольких элементов. Если такие действия выполняются только вручную, пользователь быстро тратит время на техническую корректировку схемы вместо анализа ее содержания.

Поэтому в Vision Diagram предусмотрен диалоговый сценарий. Пользователь может сформулировать просьбу обычным языком: создать диаграмму, добавить объект, переименовать блок, убрать лишнюю связь или объяснить содержимое холста. При этом результат все равно остается контролируемым, потому что изменения применяются через внутреннюю модель данных приложения.

Задача разработки заключается в создании редактора, где ручная работа и интеллектуальные команды не конкурируют между собой, а образуют единый рабочий процесс. Пользователь должен иметь возможность быстро получить основу схемы, проверить ее и при необходимости доработать обычными инструментами.

1.2 Актуальность

Актуальность разработки объясняется практической потребностью в быстрых средствах визуализации. Диаграммы используются при изучении алгоритмов, проектировании программных систем, описании бизнес-процессов и подготовке технической документации. Во

всех этих случаях важна не только красота итоговой схемы, но и скорость перехода от идеи к рабочей модели.

Современные пользовательские интерфейсы постепенно переходят от набора отдельных кнопок к комбинированному взаимодействию, где часть действий можно выразить текстом. Для редактора диаграмм это особенно естественно: большинство схем сначала существует как словесное описание, и только потом превращается в набор фигур. Поддержка естественного языка позволяет использовать этот исходный текст напрямую.

Для образовательной среды ценность такого подхода заключается в снижении порога входа. Студенту не нужно сразу знать все приемы работы с графическим редактором: достаточно описать процесс, получить черновик и затем изучить, как эта структура представлена на холсте. Такой сценарий помогает быстрее перейти от абстрактного описания к наглядной модели.

Выбор веб-формата также имеет прикладное значение. Приложение запускается в браузере, а значит его проще демонстрировать на защите, использовать в учебной аудитории и развивать без привязки к конкретной операционной системе. Серверная часть при этом может оставаться локальной и обращаться к установленной модели Ollama.

По этим причинам тема работы является актуальной: она находится на пересечении графических редакторов, учебной визуализации и практического применения языковых моделей в интерфейсах.

1.3 Постановка задачи

В рамках работы необходимо разработать веб-приложение Vision Diagram для ручного и интеллектуального создания схем и таблиц. Система должна работать как самостоятельный редактор, а не только как демонстрация генерации текста: все элементы диаграммы должны оставаться редактируемыми после построения.

В редакторе должны поддерживаться основные типы объектов, реализованные в проекте: прямоугольник, круг, ромб, список и таблица. Для каждого объекта требуется хранить идентификатор, координаты на сетке, размеры, поворот и текстовое содержимое. Для списка дополнительно используется массив строк, а для таблицы - число строк, число столбцов и матрица значений ячеек.

Система должна позволять соединять объекты связями. Пользователь выбирает исходный элемент, затем целевой элемент, после чего редактор создает связь между ними. Для разных типов

схем необходимо поддерживать несколько вариантов стрелок: линию без наконечника, заполненную стрелку, пустую стрелку, стрелку от источника и двустороннюю связь.

Ключевая задача проекта связана с интеллектуальным модулем. Клиентская часть должна отправлять на сервер текстовый запрос и, при наличии объектов на холсте, контекст текущей диаграммы. Сервер должен сформировать запрос к языковой модели, получить структурированный ответ и вернуть его клиенту в виде одного из действий: `replace`, `add`, `update`, `remove` или `analyze`.

Кроме основных операций редактирования, приложение должно сохранять диаграмму в браузере, поддерживать историю `undo/redo`, импортировать и экспортировать формат `draw.io`, а также предоставлять переключение светлой и темной темы. Эти функции делают проект не только технической демонстрацией, но и пригодным для повседневного использования в учебных сценариях.

Результатом выполнения задачи должно стать работоспособное программное средство, которое можно запустить локально, показать на защите и использовать для построения редактируемых диаграмм с участием AI-помощника.

1.4 Требования к эксплуатационным характеристикам

Система рассчитана на работу в современном браузере на персональном компьютере. Клиентская часть запускается как веб-приложение, а для интеллектуального режима дополнительно требуется работающий сервер и установленная среда `Ollama`. Такой вариант развертывания подходит для локальной демонстрации и не требует отдельного настольного приложения.

Интерфейс должен реагировать на действия пользователя без заметных задержек. Выделение, перемещение, изменение размеров, поворот объектов и построение связей выполняются на клиентской стороне, поэтому базовый редактор остается доступным даже до обращения к языковой модели.

Интеллектуальный режим зависит от серверного приложения. Сервер обращается к `Ollama`, проверяет доступность моделей, принимает `JSON`-запросы и возвращает клиенту результат в заранее определенной структуре. Если модель временно недоступна или возвращает некорректные данные, приложение должно обрабатывать такую ситуацию без разрушения текущей диаграммы.

Для учебного использования важны понятные команды запуска, предсказуемое поведение основных сценариев и возможность быстро восстановить рабочее состояние. Поэтому в проекте предусмотрено сохранение диаграммы в браузере и отдельное хранение выбранной темы оформления.

1.5 Требования к качеству исходного кода

Код проекта должен быть организован так, чтобы изменения в одной части системы не ломали остальные. В Vision Diagram это особенно важно из-за разделения на несколько слоев: визуальные компоненты редактора, функции изменения объектов, глобальное хранилище состояния, импорт и экспорт draw.io, панель AI-помощника и серверный модуль взаимодействия с моделью.

Основным языком разработки выбран TypeScript. Он позволяет явно описать фигуры, связи, текстовые стили и действия интеллектуального модуля. Благодаря этому при передаче данных между клиентом и сервером легче обнаружить несогласованность полей и избежать ошибок, которые в JavaScript проявились бы только во время выполнения.

Клиентская часть разделена на страницы, виджеты, сущности, функции и общие модули. Например, холст и строка состояния находятся в widgets, фигуры и связи - в entities, AI-помощник и импорт/экспорт - в features, а типы, тема, сетка и Zustand-хранилище - в shared. Такая организация помогает быстро найти участок кода, отвечающий за конкретное поведение интерфейса.

Отдельное требование предъявляется к устойчивости AI-модуля. Ответ языковой модели нельзя считать полностью надежным, поэтому приложение нормализует типы фигур, координаты, размеры и дополнительные поля. Если ответ не удастся разобрать, сценарий не должен приводить к потере уже созданной пользователем диаграммы.

Воспроизводимость поведения обеспечивается через явное состояние shapes и edges. Каждая операция редактирования изменяет не скрытое графическое полотно, а конкретные объекты в хранилище. Это упрощает отмену и повтор действий, экспорт в XML-структуру draw.io и передачу текущего контекста на сервер.

Читаемость кода поддерживается понятными именами модулей и небольшими областями ответственности. Например, Toolbar отвечает за верхнюю панель, DiagramCanvas - за работу с холстом, AIPanel - за диалог с моделью, а diagramStore - за состояние диаграммы. Такой состав проекта проще объяснить в пояснительной записке и дальше развивать после защиты.

Архитектура должна допускать расширение. В дальнейшем можно добавить новые типы фигур, другие варианты стрелок, дополнительные команды AI, новые форматы экспорта или альтернативные модели. Поэтому в проекте выбран подход, при котором интерфейс работает с типизированной моделью диаграммы, а сервер возвращает не произвольный текст, а ограниченный набор действий.

С учетом этих требований задача дипломного проекта сводится не только к созданию видимого интерфейса, но и к разработке достаточно аккуратной программной основы, которую можно сопровождать, проверять и расширять.

2. АНАЛИТИЧЕСКАЯ ЧАСТЬ

2.1 Анализ подобных систем

Перед разработкой были рассмотрены редакторы, которые чаще всего используются для построения диаграмм. Наиболее близким по назначению является diagrams.net (draw.io): он работает в браузере, содержит большой набор фигур и поддерживает собственный XML-формат файлов [1]. При этом его основной сценарий остается ручным: пользователь сам собирает схему из элементов и настраивает их расположение.

Lucidchart ориентирован на облачную совместную работу и корпоративные интеграции. Он удобен для команд, которым нужны шаблоны, комментарии и хранение проектов в онлайн-среде. Для дипломной задачи этот продукт полезен как пример зрелого редактора, но он не показывает, как самостоятельно построить локальный контур взаимодействия с языковой моделью.

Сервисы наподобие Miro решают более широкую задачу: они дают общую доску для идей, заметок, карточек, схем и обсуждений. Такой формат хорошо подходит для мозгового штурма, но для проекта Vision Diagram важнее не бесконечная доска, а управляемая модель диаграммы, где каждый объект имеет тип, координаты и связи.

Microsoft Visio и похожие настольные редакторы обладают мощными средствами проектирования, но для учебного прототипа они избыточны. Они требуют отдельной установки или лицензии и не дают возможности показать собственную реализацию AI-взаимодействия на уровне исходного кода.

Сравнение показало, что существующие решения хорошо закрывают отдельные стороны задачи: ручное построение, шаблоны, совместную работу или файловый обмен. Однако для дипломного проекта важна другая комбинация свойств:

- облегченный веб-интерфейс без избыточных функций;
- одновременная поддержка схем и таблиц;
- возможность генерации диаграммы по текстовому описанию;
- возможность редактирования уже существующей схемы через естественный язык;
- строгий структурированный обмен между интерфейсом и интеллектуальным модулем.

Поэтому разработка Vision Diagram не рассматривается как попытка заменить diagrams.net, Lucidchart или Visio. Проект занимает более узкую нишу: быстрый локальный редактор, где

пользователь может соединить ручное построение схемы с текстовыми командами к AI-помощнику.

2.1.1 Критерии сравнения существующих решений

Критерии сравнения выбирались исходя из реального сценария работы Vision Diagram. Оценивались не только количество фигур или наличие шаблонов, но и то, насколько инструмент удобен для быстрого запуска, учебной демонстрации и перехода от текстового описания к редактируемой диаграмме.

К ключевым критериям сравнения были отнесены:

- поддержка ручного создания и редактирования диаграмм;
- наличие табличных элементов или близких по смыслу структур;
- простота запуска и использования в браузере;
- возможность обмена файлами с другими средствами;
- наличие средств интеллектуальной или полуавтоматической генерации;
- удобство применения в учебной среде;
- степень избыточности интерфейса для локальных задач построения схем.

Такой набор критериев соответствует масштабу проекта. Vision Diagram не претендует на роль корпоративной платформы моделирования, зато должен быстро запускаться локально, иметь понятный интерфейс и давать возможность управлять диаграммой через естественный язык.

2.1.2 Сравнение с diagrams.net

diagrams.net является ближайшим функциональным ориентиром, потому что использует веб-интерфейс и формат draw.io [1], [2]. Его сильная сторона - зрелый набор инструментов: фигуры разных нотаций, шаблоны, импорт, экспорт и интеграции с облачными хранилищами. Именно поэтому в Vision Diagram выбран совместимый направление обмена файлами.

Ограничение diagrams.net для рассматриваемой задачи состоит не в недостатке функциональности, а наоборот в ее широте. Для быстрой учебной схемы интерфейс может оказаться перегруженным, а построение по текстовому описанию все равно требует ручного переноса элементов. Vision Diagram сознательно оставляет меньший набор объектов, но добавляет сценарий генерации и изменения через текстовую команду.

В сравнении с diagrams.net разработанная программа уступает по количеству готовых нотаций, но демонстрирует другой принцип работы. Пользователь может начать не с поиска

нужной фигуры в библиотеке, а с описания схемы. После генерации результат остается обычной диаграммой, которую можно двигать, редактировать, соединять и экспортировать.

2.1.3 Сравнение с Lucidchart и Miro

Lucidchart и Miro особенно сильны в совместной работе: они позволяют нескольким пользователям обсуждать схему, оставлять комментарии и использовать готовые шаблоны. Для бизнеса это полезно, но в рамках дипломного проекта главный интерес состоит не в облачной кооперации, а в собственной реализации редактора и AI-контура.

Кроме того, такие сервисы обычно скрывают внутреннюю архитектуру от пользователя. Студент может применить готовый инструмент, но не показать, как устроена модель данных, как формируется запрос к языковой модели и как ответ превращается в изменения на холсте. Vision Diagram, напротив, позволяет рассмотреть все эти этапы на уровне исходного кода.

Именно поэтому ценность Vision Diagram заключается не только в пользовательском интерфейсе, но и в прозрачности реализации. В дипломе можно проследить путь данных от кнопки на панели и SVG-объекта до Zustand-хранилища, серверного маршрута и ответа модели.

2.1.4 Сравнение с Microsoft Visio и настольными редакторами

Microsoft Visio является примером профессионального настольного редактора с богатым набором нотаций. Он подходит для крупных инженерных и корпоративных задач, но слишком тяжел для локального учебного эксперимента, где требуется показать компонентную веб-архитектуру и обработку AI-команд.

Для данной ВКР важно, чтобы архитектура оставалась наблюдаемой и объяснимой. Поэтому выбран легкий веб-подход: клиент отвечает за интерактивный холст и состояние, а сервер - за обращение к модели и нормализацию результата. Такой вариант проще развернуть, проверить и доработать в рамках дипломного проекта.

2.1.5 Итоги сравнительного анализа

Итог сравнительного анализа можно сформулировать следующим образом: существующие редакторы сильны как готовые продукты, но не закрывают именно учебно-исследовательский сценарий, где нужно разработать собственный инструмент с понятной внутренней моделью и AI-управлением. Vision Diagram сочетает:

- локальный запуск;
- легковесный веб-интерфейс;
- поддержку ручного графического редактирования;

- интеграцию с языковой моделью;
- обмен диаграммами через совместимый формат.

Выбранное направление разработки поэтому является обоснованным. Проект не копирует крупные редакторы диаграмм, а исследует более конкретную задачу: как встроить языковую модель в управляемый графический редактор без потери ручного контроля над результатом.

2.2 Анализ разрабатываемого решения

Разрабатываемое средство ориентировано на смешанный сценарий. Пользователь может построить схему полностью вручную, начать с AI-генерации или поочередно использовать оба режима. Такая логика отражена и в интерфейсе: панель AI не заменяет холст, а открывается рядом с ним и работает с тем же состоянием диаграммы.

Основой решения является формализованная модель данных. Фигуры хранятся в массиве `shapes`, связи - в массиве `edges`. У каждого объекта есть идентификатор и набор параметров, достаточных для повторной отрисовки и экспорта. Благодаря этому сервер и клиент обмениваются не произвольным описанием картинки, а структурой, которую можно проверить и применить программно.

Такой обмен снижает риск неоднозначности. Если модель должна добавить элемент, она возвращает действие `add` и описание новых фигур. Если требуется изменить подпись или размер, используется `update`. Если пользователь просит только объяснить схему, возвращается `analyze` без изменения состояния. В результате интерфейс остается предсказуемым даже при использовании вероятностной модели.

В браузере реализован полноценный набор редакторских действий: выбор объектов, перемещение по сетке, изменение размеров, поворот, множественное выделение, соединение фигур, редактирование таблиц и списков. AI-модуль добавляется к этому набору как ускоритель, а не как единственный способ работы.

Разрабатываемое решение поэтому находится между классическим графическим редактором и интеллектуальным помощником. Практическая ценность проекта состоит в том, что пользователь сохраняет контроль над диаграммой, но получает более быстрый старт и поддержку при типовых изменениях структуры.

2.3 Выбор архитектуры

2.3.1 Выбор архитектуры приложения

Для Vision Diagram выбрана клиент-серверная архитектура. Клиентская часть отвечает за интерактивный интерфейс: SVG-холст, панель инструментов, строку состояния, локальную историю изменений и сохранение диаграммы в браузере. Серверная часть изолирует работу с языковой моделью: принимает запрос, формирует системную инструкцию, вызывает Ollama и возвращает структурированный результат.

Полностью клиентский вариант был бы проще по числу процессов, но хуже подходит для контроля AI-ответа. В браузере сложнее централизованно описать системную инструкцию, обработать недоступность модели и скрыть детали интеграции. Отдельный сервер делает эту часть проекта более управляемой и расширяемой.

2.3.2 Выбор архитектуры клиентской части

Клиентская часть реализована как одностраничное React-приложение. Такой формат позволяет обновлять холст без перезагрузки страницы и мгновенно отражать изменения состояния. Компонентная структура удобна для редактора: панель инструментов, холст, строка состояния, AI-панель и отдельные фигуры могут развиваться независимо.

В кодовой базе используется функциональное разделение. В `entities` находятся фигуры и связи, в `features` - режим соединения, AI-помощник, изменение размеров, поворот, импорт и экспорт, в `widgets` - составные части интерфейса, а в `shared` - общие типы, тема, сетка и хранилище. Такая структура хорошо соответствует фактическому составу проекта.

2.3.3 Выбор архитектуры серверной части

Серверная часть представляет собой небольшой REST-сервис на Express. В нем описаны служебные маршруты `/health` и `/models`, а также основной маршрут `/generate-diagram`, который принимает пользовательский запрос и контекст текущей диаграммы.

Выделение серверного приложения дает возможность постепенно наращивать функционал. В будущем сюда можно вынести автокомпоновку, журналирование запросов, проверку целостности схемы или поддержку нескольких моделей, не переписывая клиентский редактор.

2.4 Выбор технологий

2.4.1 Выбор языка программирования

TypeScript выбран как единый язык для клиентской и серверной частей. В проекте это особенно важно, потому что диаграмма описывается не свободным набором полей, а конкретными типами: `shape`, `edge`, `arrowType`, `textStyle` и `diagram action`. Типизация помогает удерживать одинаковое понимание этих сущностей в разных слоях приложения.

При добавлении новых возможностей TypeScript сразу показывает, какие места кода нужно изменить. Например, новый тип фигуры должен появиться в модели данных, в рендеринге, в панели инструментов, в нормализации AI-ответа и в экспорте. Для дипломного проекта такая прослеживаемость важна не меньше, чем сама работоспособность.

2.4.2 Выбор технологий клиентской части

Интерфейс построен на React [3]. Эта библиотека удобна для приложения, состоящего из множества повторно используемых элементов: `Toolbar`, `DiagramCanvas`, `StatusBar`, `AIPanel`, `ShapeRenderer`, `EdgeEl`, `TableEl` и `ListEl`. Изменение состояния автоматически приводит к обновлению нужных частей интерфейса.

Vite используется как инструмент сборки и разработки [5]. Для состояния выбран Zustand [8], поскольку он позволяет компактно хранить массивы фигур и связей, историю `undo/redo`, методы изменения объектов и данные, сохраняемые в браузере. Дополнительно через отдельное Zustand-хранилище сохраняется выбранная тема оформления.

Рабочая область реализована средствами SVG. Это упрощает работу с отдельными графическими объектами, маркерами выбора, стрелками и сеткой. Кроме того, SVG хорошо сочетается с форматом `draw.io`: при экспорте данные преобразуются в XML-структуру `mxfile`, `mxGraphModel` и `mxCell`.

2.4.3 Выбор технологий серверной части

Серверная часть разработана на Node.js [6] и Express [7]. Такой стек хорошо подходит для компактного API: сервер принимает JSON, разрешает CORS-запросы от клиентского приложения, проверяет доступность Ollama и возвращает данные в формате, который клиент может сразу применить к диаграмме.

Для текущего масштаба проекта не требуется сложная серверная инфраструктура с базой данных и многослойной авторизацией. Express позволяет оставить сервер понятным, но при необходимости расширить его отдельными сервисами и маршрутами.

2.4.4 Выбор платформы для работы с языковой моделью

Для работы с языковой моделью используется Ollama [9]. Этот выбор связан с тем, что дипломный проект должен уверенно демонстрироваться локально. Сервер обращается к установленной модели, получает ответ и приводит его к JSON-формату действий над диаграммой.

Локальный запуск модели удобен и с исследовательской точки зрения. Разработчик контролирует системную инструкцию, видит список доступных моделей через серверный маршрут и может менять параметры интеграции без зависимости от внешнего облачного сервиса.

2.4.5 Выбор инструментов разработки

Для разработки используются npm, Git, ESLint и редактор кода. Этот набор не является самостоятельной особенностью проекта, но он обеспечивает воспроизводимую установку зависимостей, проверку качества кода и удобное сопровождение клиентской и серверной частей.

2.4.6 Обоснование выбора SVG как основы графического редактора

При выборе технологии визуализации учитывался характер редактора. Vision Diagram должен отображать фигуры, связи, таблицы, списки, маркеры выделения, рамку множественного выбора и управляющие элементы для изменения размеров и поворота. Для такого набора задач важна не только производительность, но и прозрачная связь между объектом данных и его представлением на экране.

SVG оказался подходящим вариантом [10], потому что каждый элемент можно представить как отдельный объект в дереве интерфейса. Это упрощает обработку щелчков, выделения, портов соединения и изменения визуального состояния. Для дипломного проекта, где количество объектов умеренное, такая ясность реализации важнее максимальной скорости отрисовки.

Canvas мог бы быть выгоднее при тысячах элементов, но потребовал бы самостоятельно реализовывать попадание курсора в объект, редактирование текста и синхронизацию с React-компонентами. В Vision Diagram эти задачи проще решаются через SVG и типизированное состояние объектов.

Поэтому выбор SVG соответствует масштабу ВКР: он делает редактор понятным для анализа, хорошо сочетается с React и позволяет напрямую объяснить связь между моделью данных и видимой диаграммой.

2.4.7 Обоснование выбора Zustand для хранения состояния

Интерактивный редактор нуждается в едином состоянии. В Vision Diagram в нем хранятся фигуры, связи, прошлые и будущие снимки для undo/redo, а также методы добавления, изменения и удаления объектов. Если распределить эти данные по множеству локальных состояний компонентов, поддерживать согласованность было бы сложнее.

Zustand выбран как компактное решение для такого сценария. Он не требует большого количества служебного кода и позволяет описать хранилище как понятный набор данных и функций. Для пояснительной записки это также удобно: логику изменения диаграммы можно объяснить через один центральный модуль.

Middleware persist используется для сохранения диаграммы в браузере. Отдельно сохраняется тема оформления, что позволяет пользователю закрыть приложение и при следующем запуске увидеть привычный режим. История undo/redo реализована через снимки состояния, ограниченные по количеству, чтобы не накапливать лишние данные.

2.4.8 Причины выбора локальной языковой модели

При интеграции интеллектуального модуля рассматривались два подхода: внешний API или локально запущенная модель. Для дипломного проекта выбран второй вариант, поскольку он лучше подходит для демонстрации собственной архитектуры и не зависит от доступа к стороннему облачному сервису.

Ollama позволяет поднять модель на локальной машине, а серверное приложение - обращаться к ней из Node.js. Это делает демонстрацию более автономной: достаточно запустить клиент, сервер и сервис модели. Кроме того, разработчик может явно проверить доступность модели и вывести список доступных вариантов через маршрут /models.

При этом архитектура не закрывает путь к дальнейшему развитию. В серверной части уже можно выделить слой, который отвечает за обращение к модели, поэтому в будущем допустимо добавить альтернативные провайдеры или несколько режимов генерации без переписывания интерфейса редактора.

3. КОНСТРУКТОРСКАЯ ЧАСТЬ

3.1 Структура приложения

3.1.1 Структура клиентской части

Клиентская часть проекта размещена в каталоге vision-digram-new-version. Основные исходные файлы находятся в директории src, которая организована по функциональным областям. В частности, в проекте выделены директории entities, features, widgets, pages, shared. Такой подход позволяет отделить базовые сущности предметной области от более сложных пользовательских сценариев и общих вспомогательных модулей.

В разделе entities размещены элементы, которые непосредственно представляют диаграмму: фигуры, списки, таблицы и связи. Раздел features содержит пользовательские функции: режим соединения объектов, AI-помощник, изменение размеров, поворот, импорт и экспорт. В widgets вынесены крупные блоки интерфейса - холст, верхняя панель и строка состояния. Общие типы, тема, сетка, геометрические функции и Zustand-хранилище находятся в shared. Такое деление совпадает с тем, как пользователь воспринимает приложение: есть объекты, действия над ними и общие механизмы редактора.

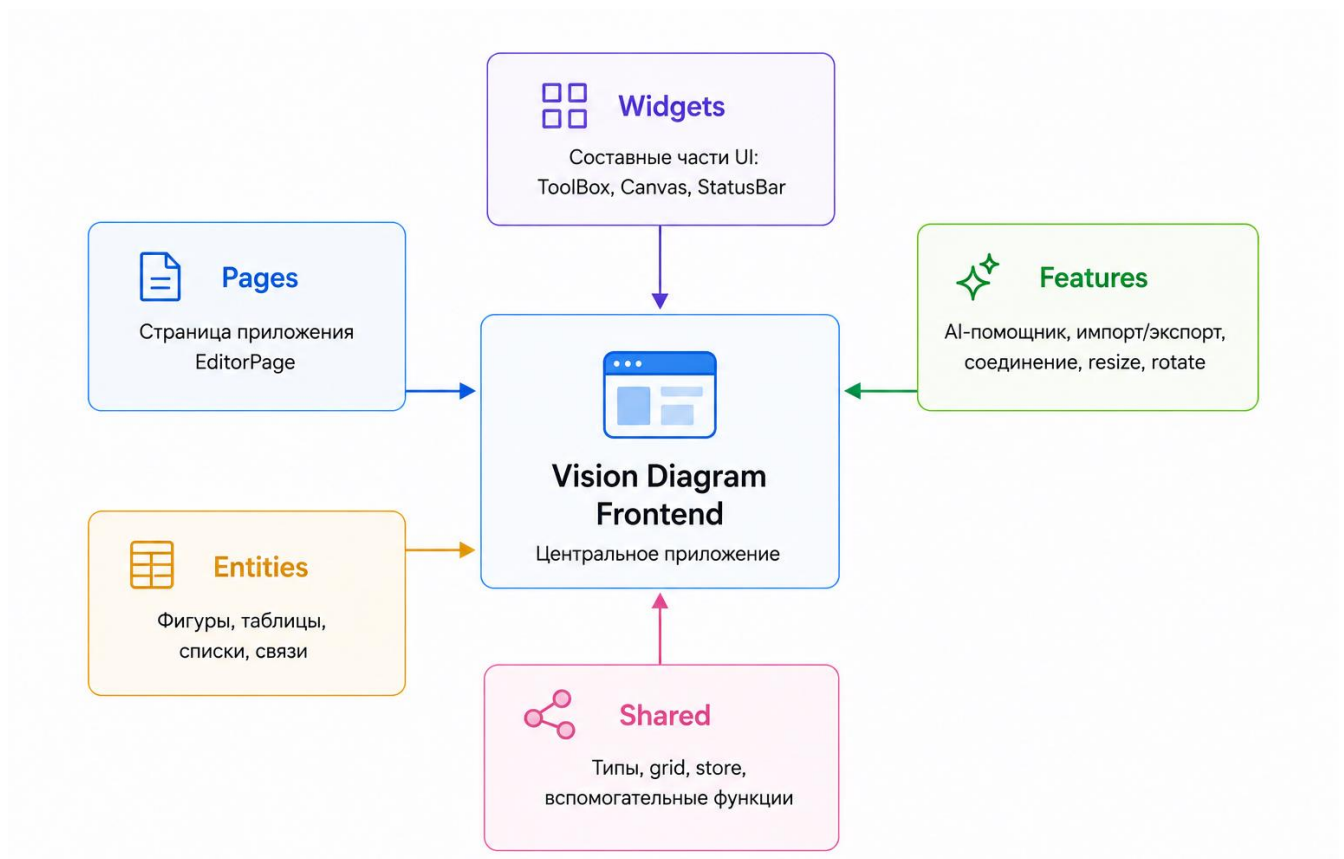


Рисунок 1. Структура клиентской части приложения Vision Diagram

3.1.2 Структура серверной части

Серверная часть проекта находится в каталоге vision-diagram-backand. Основной файл server.ts содержит настройку Express, подключение CORS и JSON-обработчика, создание клиента Ollama и описание маршрутов /health, /models и /generate-diagram. В этом же модуле заданы структуры запроса и ответа, с которыми взаимодействует клиентская часть.

Несмотря на компактный размер, сервер уже выполняет отдельную роль интеллектуального слоя. Он не занимается отрисовкой и не хранит пользовательский интерфейс, а принимает текстовый запрос, готовит инструкцию для модели, проверяет ответ и возвращает редактору безопасную структуру данных. В дальнейшем этот файл можно разделить на маршруты, сервис генерации, модуль нормализации и журналирование.

3.1.3 Взаимодействие клиентской и серверной частей

Клиент и сервер обмениваются данными по HTTP в формате JSON. Когда пользователь вводит команду в AI-панели, клиент отправляет POST-запрос на /generate-diagram. В тело запроса попадает prompt, а при наличии объектов на холсте дополнительно передается context с массивами shapes и edges.

Если холст пустой, модель получает задачу построить новую диаграмму. Если объекты уже существуют, сервер передает их как контекст, и модель должна вернуть локальное изменение: добавить, обновить, удалить или проанализировать элементы. Такой режим позволяет не терять уже выполненную пользователем ручную работу.

После ответа сервера клиент смотрит на поле action. Для replace вызывается загрузка новой диаграммы, для add новые фигуры и связи добавляются к текущим, для update меняются указанные свойства, для remove удаляются заданные идентификаторы, а analyze выводит текстовый ответ без изменения холста.

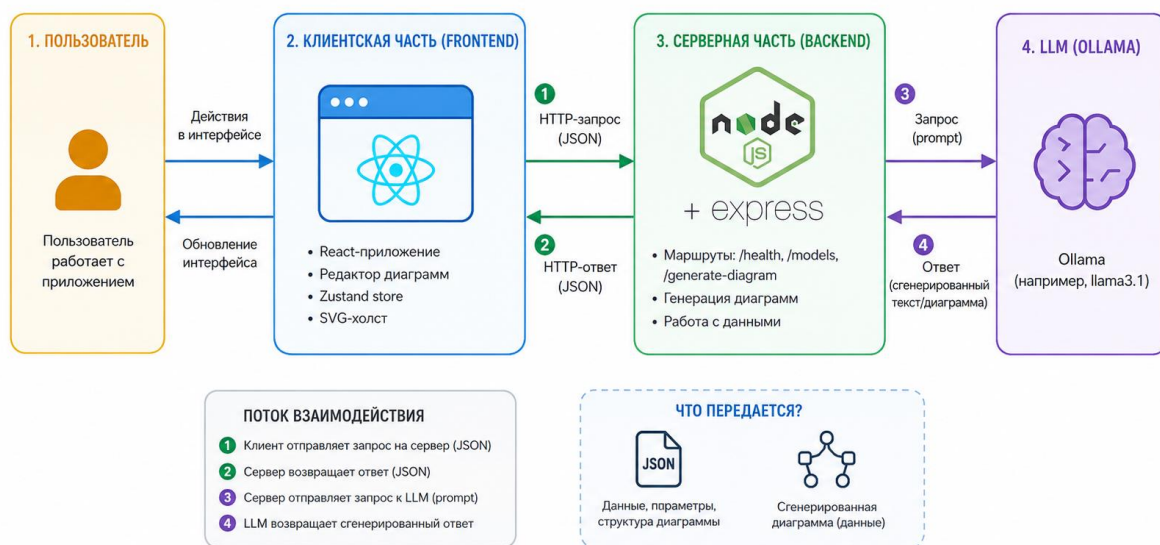


Рисунок 2. Схема взаимодействия клиентской и серверной частей

3.2 Модель данных диаграммы

Модель диаграммы состоит из двух основных массивов: shapes и edges. Фигура содержит id, type, координаты x и y, размеры w и h, текст, угол поворота и дополнительные поля для

специальных объектов. У списка это массив `items`, у таблицы - `rows`, `cols` и двумерный массив `cells`.

Координаты и размеры задаются в ячейках внутренней сетки, а не в случайных пикселях. Это упрощает выравнивание объектов, расчет положения на SVG-холсте и последующее преобразование в формат `draw.io`, где размеры должны быть переданы в XML-геометрию.

Связь содержит собственный `id`, идентификатор источника, идентификатор целевого объекта и тип стрелки. В редакторе поддерживаются `line`, `filled`, `empty`, `source` и `both`. Эти значения используются как в отображении SVG-маркеров, так и при экспорте в `draw.io`.

Для AI-взаимодействия дополнительно используется структура действия. Поле `action` определяет смысл операции, `message` содержит пояснение для пользователя, а остальные поля зависят от действия. Например, `replace` и `add` возвращают новые `shapes` и `edges`, `update` передает изменяемые свойства, `remove` - списки идентификаторов, а `analyze` оставляет диаграмму без изменений.

Выбранная модель данных обладает следующими преимуществами:

- она достаточно проста для сериализации и передачи по сети;
- она пригодна для валидации и нормализации ответа модели;
- она поддерживает как ручное редактирование, так и генерацию по текстовому описанию;
- она допускает дальнейшее расширение состава поддерживаемых элементов.

Единообразие представления данных на клиенте и сервере снижает вероятность ошибок. Если клиент ожидает объект `table` с `rows`, `cols` и `cells`, сервер либо возвращает такую структуру, либо нормализует ответ модели до допустимого вида. Это особенно важно при работе с языковой моделью, которая может допустить неточное имя поля или слишком маленький размер объекта.

В клиентской модели также предусмотрен `textStyle`. Он включает шрифт, размер, жирность, курсив, подчеркивание, выравнивание, цвет и межстрочный интервал. Даже если часть этих настроек пока не вынесена в отдельную пользовательскую панель, сама модель уже подготовлена к дальнейшему расширению возможностей оформления.

Жизненный цикл объекта начинается либо с ручного добавления через панель инструментов, либо с ответа AI-модуля. Затем объект попадает в Zustand-хранилище, отображается на холсте и может изменяться пользователем. Значимые изменения сохраняются в истории, а при экспорте текущее состояние преобразуется в XML-файл `draw.io`.

Плоская сериализуемая модель выбрана сознательно. Ее легко превратить в JSON, отправить на сервер, использовать как контекст для модели и восстановить обратно в браузере.

Чем меньше скрытых зависимостей между объектами, тем надежнее работают импорт, экспорт, undo/redo и интеллектуальное редактирование.

3.2.1 Структура данных фигуры

Каждая фигура в Vision Diagram имеет набор обязательных атрибутов. К ним относятся идентификатор, тип, координаты размещения, размеры и текстовое содержимое. Эти свойства присутствуют у всех элементов, независимо от того, является ли фигура простым геометрическим объектом или составным элементом наподобие таблицы.

Идентификатор используется для однозначной адресации объекта в пределах диаграммы. Он особенно важен при работе интеллектуального модуля, так как обновление или удаление фигур выполняется именно по их идентификаторам. Координаты и размеры выражаются в ячейках сетки, благодаря чему обеспечивается единый масштаб для всех элементов редактора. Это также упрощает математическую обработку положения фигур на рабочей области.

Для типа list в структуре фигуры дополнительно хранится массив строк, представляющих пункты списка. Для типа table используются поля rows, cols и cells, описывающие размерность и содержимое таблицы. Такой подход позволяет использовать обобщенную модель фигуры, сохраняя при этом возможность хранить специфические данные для отдельных классов объектов.

3.2.2 Структура данных связи

Связь между фигурами представлена существенно проще, чем сама фигура, однако играет важную роль для всей системы. В структуре связи содержатся идентификатор, ссылка на исходную фигуру, ссылка на целевую фигуру и тип стрелки. Наличие типизированного поля стрелки позволяет использовать различные семантические варианты отображения связей: направленный поток, двунаправленную ассоциацию, пустую стрелку или линию без конечника.

Связи не хранят в явном виде все точки маршрута на холсте. Вместо этого путь вычисляется динамически на клиентской стороне на основе расположения фигур. Такой подход упрощает редактирование диаграммы. Если пользователь переместил элемент, то связь автоматически перестраивается без необходимости обновлять отдельные координаты промежуточных сегментов.

3.2.3 Структура действия интеллектуального модуля

Наиболее характерной частью модели данных проекта является структура действия над диаграммой. Именно она делает возможным управляемое интеллектуальное редактирование.

Вместо передачи произвольного текстового ответа сервер возвращает объект, в котором явно указано, какую именно операцию требуется выполнить.

Подход с действиями удобен по нескольким причинам. Во-первых, он снижает неоднозначность. Клиентская часть не пытается угадывать намерение модели, а ориентируется на конкретное поле action. Во-вторых, он делает поведение интерфейса проверяемым: для каждого типа действия можно описать собственную ветвь обработки и собственный набор допустимых данных. В-третьих, такая схема хорошо масштабируется и позволяет добавлять новые интеллектуальные возможности без изменения базового принципа обмена.

3.3 Реализация графического редактора

Рабочая область Vision Diagram реализована на основе SVG- холста. На фоне отображается сетка, которая помогает пользователю выравнивать элементы и визуально ориентироваться на плоскости диаграммы. Размеры диаграммы задаются количеством столбцов и строк сетки, а размещение элементов выполняется в координатах, связанных с этой сеткой.

Редактор поддерживает два основных режима добавления элементов. В обычном режиме пользователь выбирает фигуру на панели инструментов, после чего размещает ее на холсте щелчком мыши. Второй режим связан с интеллектуальным помощником, который может добавить элементы автоматически по текстовому запросу.

После создания объекта пользователь может перемещать его по рабочей области, менять размеры и поворачивать фигуру. Для этого используются специальные управляющие элементы, отображаемые при выделении объекта. Для простых фигур поддерживается редактирование подписи по двойному щелчку. Для таблиц реализовано редактирование содержимого отдельных ячеек, а также добавление и удаление строк и столбцов.

Система поддерживает режим соединения элементов. Пользователь выбирает исходную фигуру, затем указывает целевой объект. После этого в хранилище добавляется связь, а ее графическое представление вычисляется автоматически. В приложении используется алгоритм построения ортогонального пути, при котором связь состоит из горизонтальных и вертикальных сегментов. Это делает изображение более аккуратным и привычным для диаграммных нотаций.

Особое внимание уделено управлению историей изменений. В приложении реализованы операции отмены и повтора действий. Для этого в хранилище состояния поддерживаются снимки прошлого и будущего состояния диаграммы. Каждое значимое изменение фиксируется в истории, что повышает удобство работы и снижает риск потери промежуточного результата.

Состояние диаграммы сохраняется на стороне клиента, что позволяет восстанавливать рабочую область при повторном открытии приложения в браузере. Такой подход удобен для одиночного сценария эксплуатации и упрощает демонстрацию проекта.

Переключение темы не затрагивает структуру диаграммы и влияет только на визуальное представление интерфейса. Выбранный пользователем режим сохраняется в локальном хранилище браузера и автоматически восстанавливается при следующем открытии приложения. Это повышает эргономичность системы и позволяет адаптировать рабочую среду под разные условия освещения и индивидуальные предпочтения пользователя.

Дополнительным элементом удобства стала подсистема визуальных тем. В редакторе реализованы две палитры оформления - темная и светлая, а их параметры описаны единым набором цветовых переменных. Такой подход обеспечивает единообразное оформление верхней панели, строки состояния, рабочего холста, фигур, таблиц, элементов выделения и управляющих маркеров.

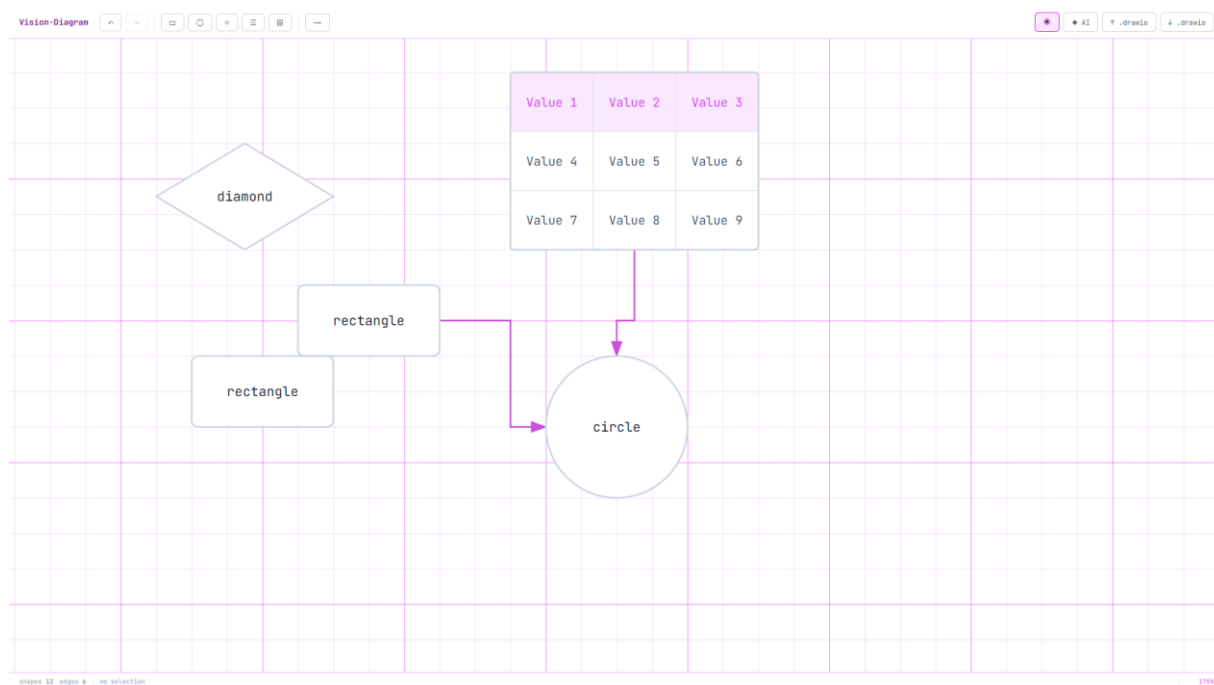


Рисунок 3. Рабочая область редактора с размещенными фигурами

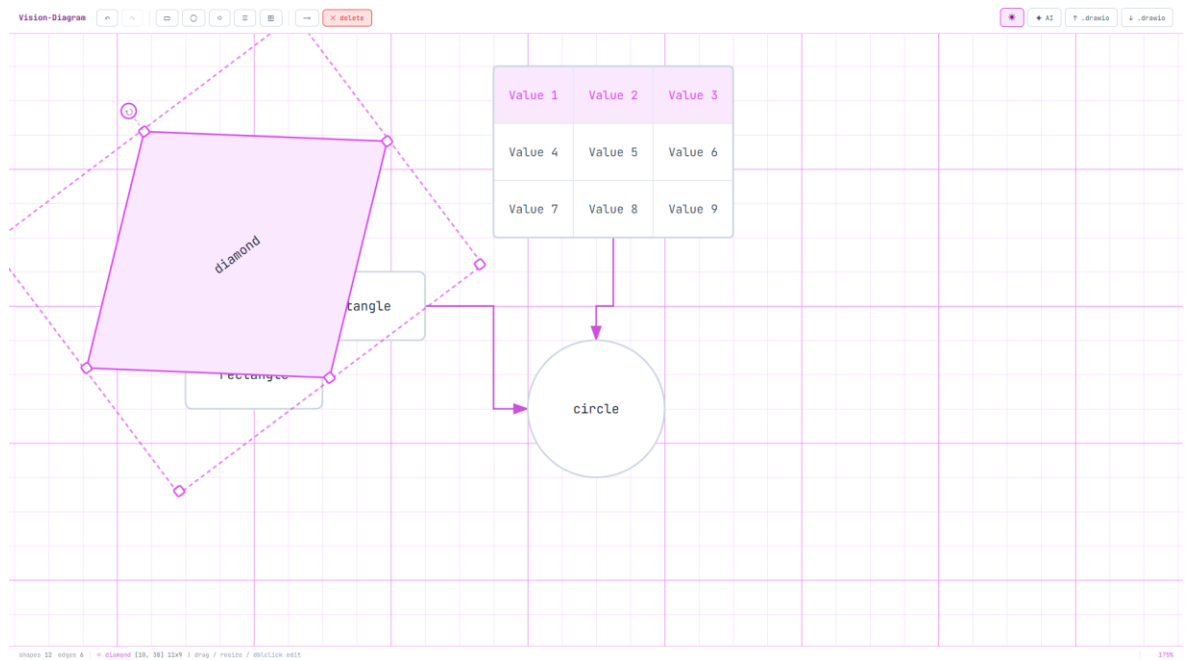


Рисунок 4. Пример выделения, изменения размеров и поворота фигуры

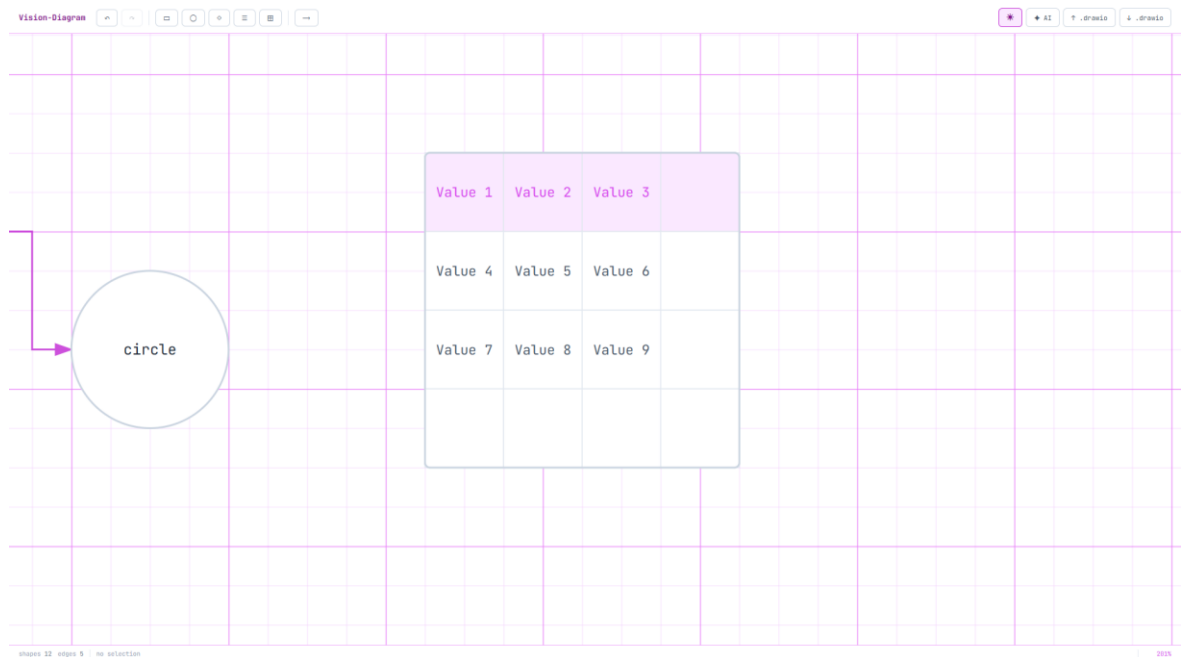


Рисунок 5. Редактирование таблицы в рабочей области

3.4 Реализация интеллектуального модуля

Интеллектуальный модуль является одной из ключевых особенностей Vision Diagram. На клиентской стороне он представлен боковой панелью, в которой пользователь вводит текстовый запрос и получает ответ системы в формате диалога. Панель отображает историю сообщений и

позволяет отправлять команды как на создание новой диаграммы, так и на изменение уже существующей.

При отправке запроса клиент сначала проверяет, есть ли на холсте объекты. Если массивы `shapes` и `edges` пустые, запрос воспринимается как создание новой диаграммы. Если элементы уже есть, клиент прикладывает их к запросу как `context`. Поэтому модель получает не только фразу пользователя, но и текущее состояние рабочей области.

На сервере формируется системная инструкция для модели. В ней перечисляются допустимые типы фигур, варианты стрелок, ограничения сетки, требования к JSON-ответу и набор действий, которые клиент умеет выполнять:

- `replace` - полная замена диаграммы;
- `add` - добавление новых элементов;
- `update` - изменение свойств существующих фигур;
- `remove` - удаление элементов;
- `analyze` - текстовое описание текущего содержимого без изменения диаграммы.

После получения ответа сервер пытается разобрать JSON и привести данные к допустимому виду. Проверяются типы фигур, координаты, размеры, массивы элементов списка и ячейки таблицы. Если ответ не подходит для применения, сервер возвращает безопасный результат с сообщением об ошибке, не разрушая текущую работу пользователя.

Клиент применяет ответ через методы хранилища. `replace` загружает новую диаграмму, `add` добавляет элементы к текущим, `update` изменяет только переданные свойства, `remove` удаляет указанные объекты и связи, а `analyze` пополняет историю сообщений без изменения холста.

Таким образом, AI-модуль не вмешивается напрямую в отрисовку. Он действует через формальную модель операций, а значит результат можно отменить, проверить, экспортировать и доработать вручную обычными средствами редактора.

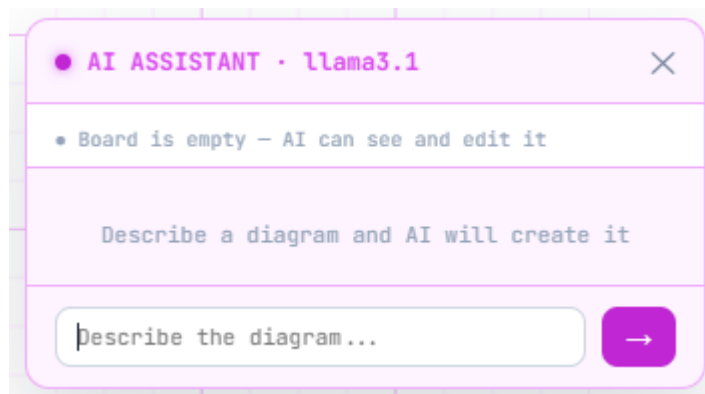


Рисунок 6. Панель интеллектуального помощника Vision Diagram

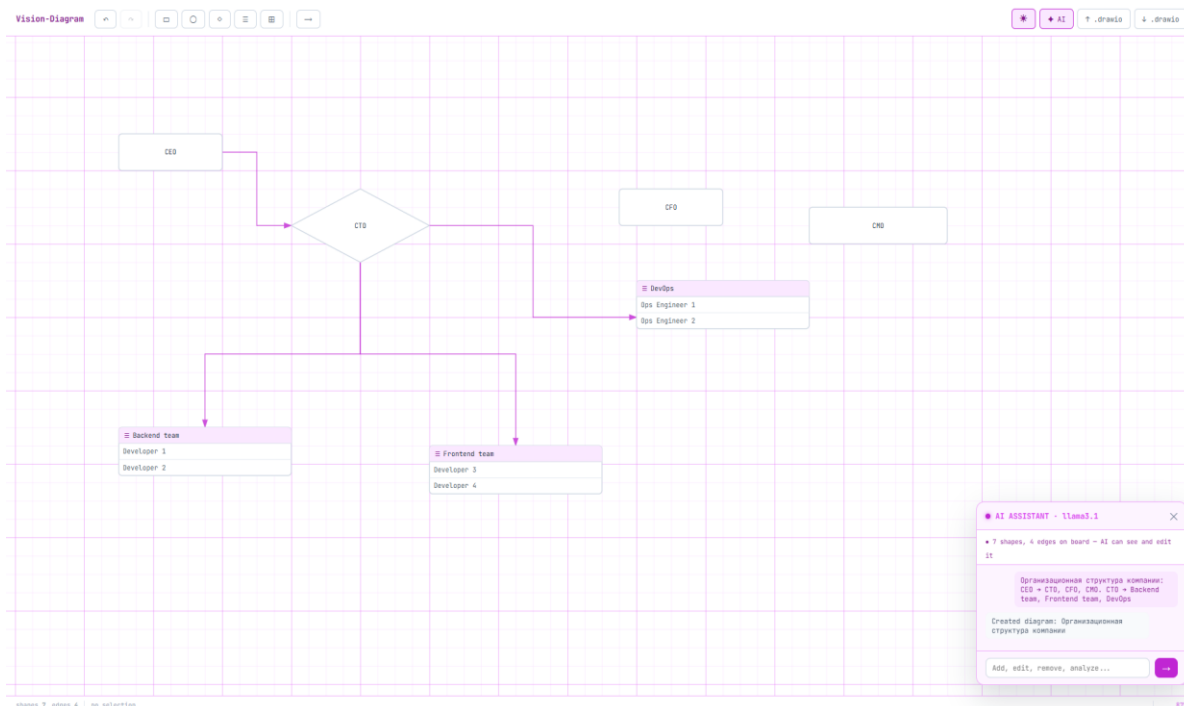


Рисунок 7. Результат генерации схемы по текстовому описанию

3.5 Импорт и экспорт диаграмм

Для повышения практической полезности приложения реализована поддержка импорта и экспорта в формате draw.io. Экспорт выполняется путем преобразования внутренних структур фигур и связей в XML-представление `mxGraphModel`. Для каждого типа объекта формируется соответствующий XML-фрагмент. Простые фигуры экспортируются как стандартные ячейки графа, списки представляются через контейнеры с дочерними элементами, а таблицы - через структуру строк и вложенных ячеек.

При импорте приложение выполняет обратное преобразование. XML-файл разбирается средствами браузера, после чего из него извлекаются вершины и связи. На основе стилевых признаков определяется тип фигуры, восстанавливаются координаты, размеры, поворот и содержимое. Для таблиц и списков выполняется дополнительная реконструкция внутренней структуры.

Поддержка импорта и экспорта делает систему более совместимой с внешними инструментами. Пользователь может сохранить результат работы в понятном формате и открыть его в стороннем редакторе или, наоборот, загрузить существующую диаграмму в Vision Diagram для дальнейшего интеллектуального редактирования. Структура обмена базируется на представлении `mxGraphModel`, используемом в экосистеме draw.io [2].

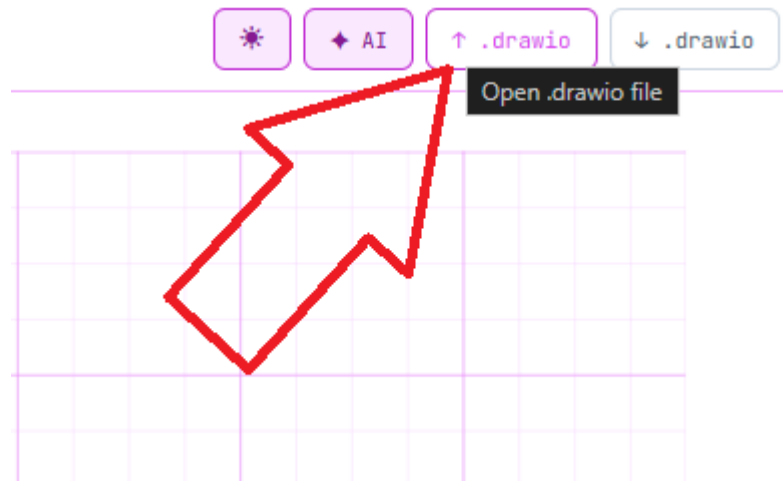


Рисунок 8. Импорт диаграммы из файла `draw.io`

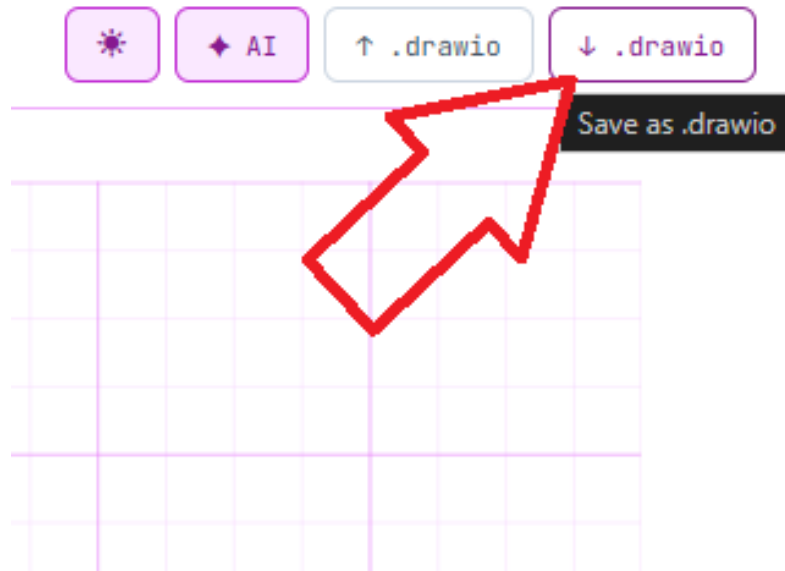


Рисунок 9. Экспорт диаграммы в формат `draw.io`

Важно отметить, что реализация импорта и экспорта не ограничивается простым сохранением текста в файл. Фактически приложение выполняет двустороннее преобразование между собственной внутренней моделью диаграммы и одной из распространенных внешних форм ее представления. Для дипломного проекта это существенный плюс, так как система становится не замкнутым прототипом, а инструментом, способным обмениваться данными с другими редакторами.

С точки зрения архитектуры данная возможность также полезна тем, что подтверждает корректность выбранной модели данных. Если внутренние структуры фигур и связей оказываются достаточно выразительными, чтобы быть экспортированными в XML-граф и затем

обратно импортированными с восстановлением ключевых характеристик, значит выбранное представление действительно пригодно для практического использования.

3.6 Пользовательские сценарии работы

В рамках проектирования системы можно выделить несколько основных пользовательских сценариев.

Первый сценарий - ручное создание диаграммы. Пользователь запускает приложение, выбирает нужный тип фигуры на панели инструментов, размещает объект на холсте, редактирует подписи и соединяет элементы связями.

Второй сценарий - создание схемы по текстовому описанию. Пользователь открывает интеллектуальную панель и формулирует запрос, например описывает структуру алгоритма или набор взаимосвязанных компонентов. После этого система генерирует диаграмму и отображает ее на рабочей области.

Третий сценарий - изменение уже существующей схемы. Пользователь может попросить систему добавить элемент, удалить группу фигур, изменить подписи или проанализировать содержимое рабочей области. Такой режим особенно удобен при итеративной доработке диаграммы.

Четвертый сценарий - работа с таблицей. Пользователь создает табличный объект, изменяет количество строк и столбцов, редактирует значения ячеек и использует таблицу как часть общей диаграммы.

Пятый сценарий - обмен данными с внешними средствами. Пользователь импортирует файл draw.io, редактирует его в Vision Diagram, а затем экспортирует результат обратно в совместимый формат.

Шестой сценарий связан с поэтапной доработкой диаграммы. В реальной практике пользователь редко получает итоговую схему с первой попытки. Чаще процесс работы выглядит как последовательность уточнений: сначала формируется черновая структура, затем корректируются подписи, добавляются новые связи, удаляются лишние элементы и перерабатывается компоновка. За счет наличия истории изменений и интеллектуального помощника Vision Diagram хорошо подходит именно для такой итеративной работы.

Седьмой сценарий характерен для учебных демонстраций и лабораторных работ. Преподаватель или студент может быстро сформировать исходную схему по текстовому описанию, а затем использовать ее как заготовку для объяснения темы. Например, можно

показать начальную блок-схему алгоритма и постепенно изменять ее в процессе разбора материала. Это делает приложение полезным не только как инструмент проектирования, но и как средство визуального сопровождения учебного процесса.

Сценарный анализ подтверждает, что Vision Diagram ориентирован на широкий круг задач при сохранении достаточно компактного интерфейса. Пользователь может использовать приложение как традиционный редактор, как интеллектуальный генератор схем или как промежуточный инструмент между этими режимами.

3.7 Тестирование программного средства

Тестирование является обязательным этапом разработки программного средства, особенно если система сочетает пользовательский интерфейс, графическое редактирование и обращение к языковой модели. Для Vision Diagram тестирование преследовало несколько целей: подтвердить корректность базовых пользовательских сценариев, убедиться в работоспособности клиентской и серверной частей, а также проверить устойчивость системы к типовым ошибкам ввода и неидеальным ответам интеллектуального модуля.

В рамках дипломного проекта применялись главным образом методы функционального и интеграционного тестирования. Это объясняется характером системы: ключевой интерес представляет не отдельная изолированная функция, а корректное взаимодействие всех компонентов в реальном пользовательском сценарии. Также были проведены проверки сборки и типизации проекта, что позволяет рассматривать их как элементы технического контроля качества.

3.7.1 Цели и задачи тестирования

Основными задачами тестирования являлись:

- проверка корректного запуска клиентской и серверной частей;
- проверка доступности маршрутов серверного приложения;
- проверка операций создания, удаления, перемещения и редактирования фигур;
- проверка работы таблиц и списков как составных элементов;
- проверка создания и изменения связей между объектами;
- проверка истории изменений undo/redo;
- проверка импорта и экспорта диаграмм;
- проверка интеллектуальной генерации новой схемы;
- проверка интеллектуального изменения уже существующей диаграммы.

Каждая из этих задач отражает отдельный аспект системы. Например, проверка запуска подтверждает корректность окружения и зависимостей, а тестирование интеллектуального модуля показывает, насколько надежно клиент и сервер работают с ответами модели в рамках формализованной структуры действий.

3.7.2 Проверка клиентской части

Клиентская часть была проверена как на уровне сборки, так и на уровне пользовательских сценариев. При технической проверке использовалась команда производственной сборки, которая выполняет типизацию и подготовку клиентского приложения к публикации. Успешное завершение данной команды подтверждает отсутствие критических ошибок TypeScript и пригодность проекта к развертыванию в собранном виде.

Проверка интерфейса выполнялась как последовательность пользовательских действий в браузере. Создавались прямоугольники, круги, ромбы, списки и таблицы, после чего проверялись выделение, перемещение, изменение размеров, поворот и удаление. Отдельно оценивалась строка состояния: она должна показывать количество объектов, режим работы и сведения о выделенной фигуре или связи.

Навигация по холсту проверялась через панорамирование, масштабирование и множественное выделение рамкой. Эти сценарии важны, потому что даже учебная диаграмма может быстро разрастись до нескольких связанных блоков, и пользователь должен сохранять контроль над всей композицией.

3.7.3 Проверка серверной части

Сервер проверялся через запуск в режиме разработки и обращение к служебным маршрутам. /health использовался для контроля запуска приложения, /models - для проверки связи с Ollama и получения списка доступных моделей. Затем тестировался основной маршрут /generate-diagram, принимающий текстовый запрос и контекст диаграммы.

Дополнительно выполнялась компиляционная проверка TypeScript. Она важна для серверной части, потому что структуры запроса, ответа и действий должны быть согласованы с клиентской логикой. Успешная проверка означает, что основные типы данных описаны без противоречий.

3.7.4 Проверка интеллектуального модуля

AI-модуль проверялся особенно внимательно, поскольку он получает данные от вероятностной модели. Обычная логика редактора полностью контролируется кодом, а здесь

возможны неточные поля, лишний текст или неподходящие размеры. Поэтому при тестировании оценивалась не только генерация, но и устойчивость нормализации.

Использовались два базовых сценария. В первом пользователь отправлял запрос на пустом холсте, и система должна была вернуть replace с новой структурой. Во втором на холсте уже была диаграмма, и модель получала ее как context. В этом случае ожидался add, update, remove или analyze, то есть локальное изменение без полной перезаписи результата.

Практическая проверка подтвердила, что интеграция с Ollama работает. При этом качество ответа зависит от конкретной формулировки запроса: четкие команды вроде «добавь блок проверки» или «построй схему обработки заказа» выполняются надежнее, чем длинные неоднозначные описания. Это ожидаемое ограничение и причина, по которой серверная нормализация является обязательной частью проекта.

3.7.5 Проверка импорта и экспорта

Импорт и экспорт проверялись на XML-представлении draw.io. При экспорте приложение создает файл с элементами mxfile, mxGraphModel и mxCell, где фигуры, таблицы, списки и связи получают собственные идентификаторы и геометрию.

При импорте проверялось восстановление внутренней модели диаграммы из внешнего файла. После загрузки элементы должны становиться обычными объектами редактора: их можно выделять, перемещать, соединять, изменять и повторно экспортировать.

3.7.6 Результаты тестирования и выводы

По результатам тестирования текущая версия Vision Diagram выполняет основные задачи ВКР. Клиентская часть запускается и позволяет редактировать диаграммы, сервер отвечает на служебные запросы, Ollama используется для генерации структурированных действий, а импорт и экспорт обеспечивают обмен с форматом draw.io.

При этом выявлены ограничения, которые важно честно учитывать. Приложение рассчитано на одиночную локальную работу, пока не содержит базы данных, аккаунтов и совместного редактирования. Генеративная модель требует ясных запросов, а набор поддерживаемых объектов ограничен базовыми фигурами, списками и таблицами. Эти ограничения не мешают демонстрации основной идеи, но задают направления дальнейшего развития.

3.7.7 Контрольные примеры запросов к интеллектуальному модулю

Контрольные запросы нужны не только для проверки факта генерации, но и для оценки того, насколько удобно пользователю управлять редактором через текст. В Vision Diagram такие запросы можно разделить на несколько групп: создание новой схемы, добавление элементов, изменение существующих объектов, удаление и анализ содержимого.

Первая группа - запросы на построение новой диаграммы. Например, пользователь может попросить создать блок-схему обработки заказа с этапами создания, проверки, оплаты и завершения. В этом случае рабочая область пуста, поэтому сервер возвращает `replace`, а клиент загружает полученные фигуры и связи как новую модель.

Вторая группа - дополнение существующей схемы. Такой режим проверяет, умеет ли система использовать `context`. Пользователь может попросить добавить блок ошибки, новую таблицу параметров или дополнительную связь. При корректном ответе текущая диаграмма не исчезает, а расширяется новыми объектами.

Третья группа связана с изменением свойств. Пользователь может попросить переименовать блок, изменить размер элемента или переместить часть схемы. Эти сценарии сложнее, потому что модели нужно ссылаться на уже существующие объекты, но именно они показывают отличие редактора от простого генератора картинок.

Четвертая группа - аналитические запросы. Пользователь просит объяснить, что находится на холсте, перечислить основные блоки или кратко описать логику процесса. В этом случае действие `analyze` не меняет данные диаграммы, а только возвращает текстовое сообщение в историю диалога.

Комбинированные запросы лучше выполнять поэтапно. Тестирование показало, что длинная команда с несколькими условиями может дать менее предсказуемый результат. Более надежный сценарий - сначала построить основу, затем отдельной командой добавить элемент и после этого вручную или через AI уточнить детали.

Такие примеры показывают практическую роль AI-помощника: он ускоряет старт и типовые правки, но не отменяет визуальный контроль пользователя. Именно это сочетание делает Vision Diagram полезным как учебный редактор диаграмм.

4. ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ

4.1 Запуск системы

Для запуска клиентской части необходимо перейти в каталог `vision-digram-new-version` и выполнить установку зависимостей, после чего запустить приложение в режиме разработки. Типовой порядок действий соответствует возможностям инструментов Node.js, npm и Vite [5], [6] и выглядит следующим образом:

```
npm install  
npm run dev
```

Для запуска серверной части необходимо открыть каталог `vision-digram-backand`, убедиться в наличии установленных зависимостей и выполнить команду запуска сервера. Сервер реализован на Node.js и Express [6], [7]:

```
npm install  
npm run dev
```

Дополнительно должен быть запущен сервис Ollama с доступной моделью, указанной в настройках сервера. После запуска клиентская часть обращается к серверу по адресу `http://localhost:5000`.

Перед первым запуском важно убедиться, что необходимые зависимости действительно установлены и версия Node.js совместима с используемыми пакетами. Для клиентской части проект рассчитан на современную среду выполнения JavaScript, а для серверной части также требуется корректная работа TypeScript и ts-node-dev. На практике перед началом эксплуатации рекомендуется выполнить установку зависимостей отдельно в каталоге фронтенда и отдельно в каталоге бэкенда.

Дополнительной частью подготовки является проверка доступности Ollama. Если локальная модель не запущена, серверная часть может стартовать, но интеллектуальные функции не будут возвращать корректный результат. Поэтому в технической документации целесообразно рассматривать запуск системы как последовательность из трех действий: установка зависимостей, запуск локальной модели и запуск двух программных частей проекта.

4.2 Работа с редактором

После открытия приложения пользователь видит панель инструментов, рабочую область и строку состояния. Для добавления фигуры необходимо выбрать нужный тип на панели инструментов и щелкнуть по холсту. После размещения фигура становится частью диаграммы и может быть отредактирована.

Выделенный объект можно перемещать мышью. Также доступны изменение размеров через маркеры и поворот через отдельный управляющий элемент. Для изменения текста простой фигуры используется двойной щелчок. Для таблицы двойной щелчок выполняется по конкретной ячейке.

Для удаления выделенного объекта используется клавиша Delete или соответствующая команда интерфейса. Для отмены и повтора действий поддерживаются сочетания Ctrl+Z и Ctrl+Shift+Z. Масштаб рабочей области можно изменять с помощью клавиатуры или колеса мыши при нажатой клавише Ctrl.

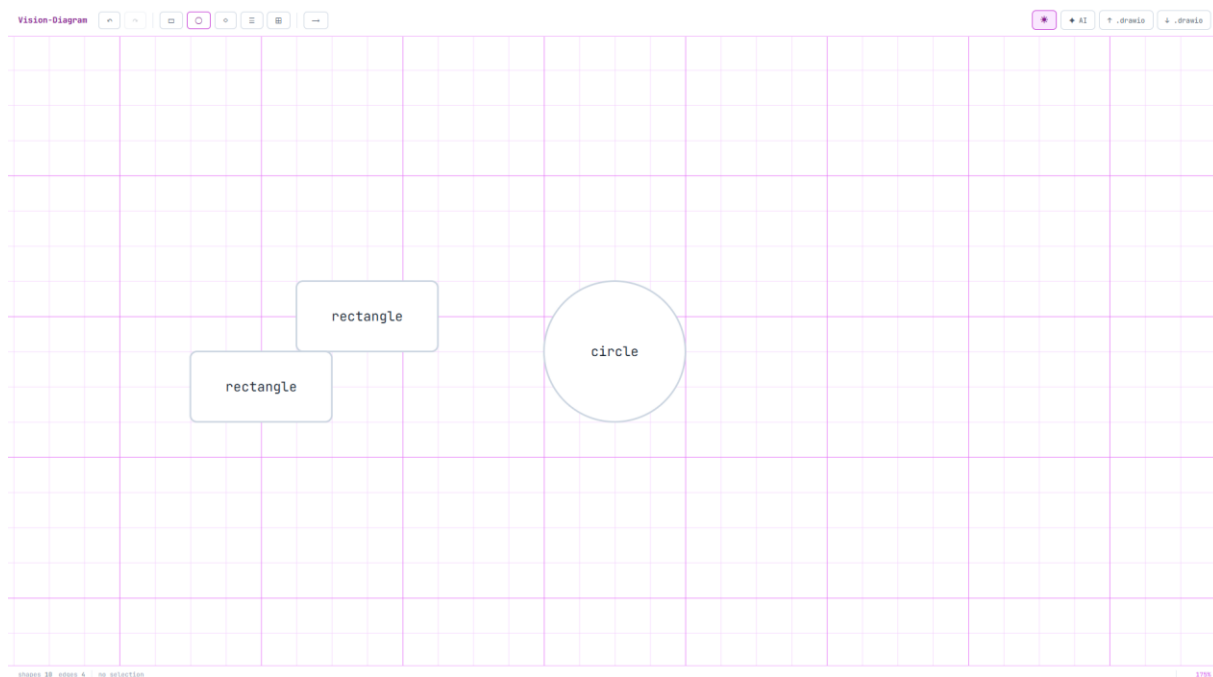


Рисунок 10. Добавление фигуры на холст

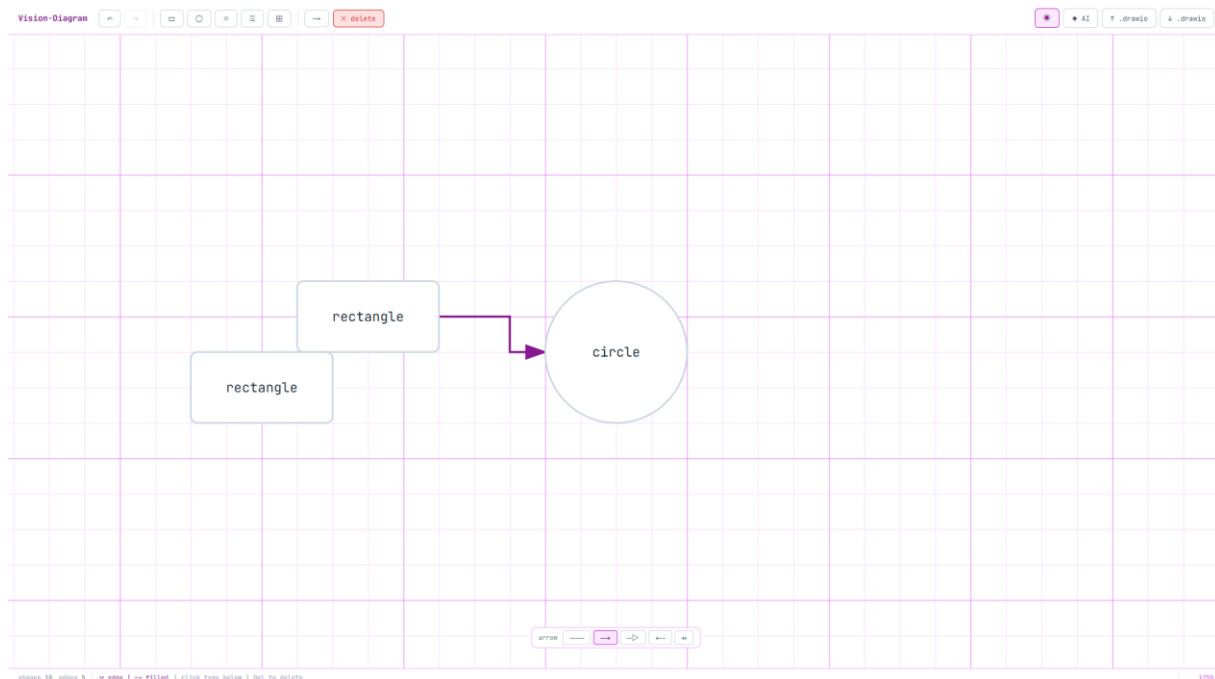


Рисунок 11. Соединение двух объектов связью

Работа с редактором начинается с выбора подходящего типа объекта на верхней панели инструментов. После этого пользователь переходит в режим размещения фигуры и указывает ее положение на рабочем поле. Такой двухэтапный механизм полезен тем, что позволяет сначала выбрать семантику будущего элемента, а затем определить его место в композиции диаграммы.

Для простых фигур редактирование текста выполняется по двойному щелчку. Это делает взаимодействие интуитивным и не требует открытия дополнительных диалоговых окон. Для таблицы реализован сходный принцип, но уже на уровне конкретной ячейки. Такой подход соответствует ожидаемому поведению табличного интерфейса и позволяет использовать таблицы как полноценный рабочий элемент схемы.

Если пользователю необходимо построить более сложную диаграмму, он может комбинировать типы объектов. Например, прямоугольники подходят для операций и модулей, ромбы - для условий и ветвлений, круги - для начальных и конечных состояний, список - для перечисления сущностей, а таблица - для представления структурированных параметров. Благодаря этому даже текущий набор типов покрывает значительную часть учебных и инженерных сценариев.

4.2.1 Общая компоновка пользовательского окна

Интерфейс Vision Diagram организован вокруг трех постоянных зон. Сверху находится панель инструментов, в центре - SVG-холст с сеткой, снизу - строка состояния. Такое расположение выбрано потому, что пользователь во время построения схемы должен одновременно видеть инструменты, саму диаграмму и текущий режим работы.

Верхняя панель отвечает за быстрые действия, центральная область - за редактирование, а строка состояния - за обратную связь. В результате пользователю не нужно постоянно открывать дополнительные меню, чтобы понять, какая фигура выбрана, какой масштаб установлен или активен ли режим соединения объектов.

AI-панель также не переводит пользователя на отдельный экран. Она появляется как фиксированный блок в правой нижней части окна, поэтому диалог с моделью происходит рядом с диаграммой. Это важно: пользователь видит результат команды сразу и может немедленно исправить его ручными средствами.

4.2.2 Верхняя панель инструментов и назначение кнопок

Верхняя панель является главным центром управления. На ней расположены название программы, кнопки undo/redo, инструменты добавления фигур, режим соединения, удаление выбранного объекта, переключатель темы, вызов AI-помощника и команды импорта и экспорта .drawio.

Логотип Vision Diagram расположен слева и выполняет роль ориентира. Для пользователя это простая подпись приложения, а для дипломной демонстрации - признак того, что перед комиссией показывается законченный программный продукт с собственным интерфейсом.

Сразу после логотипа располагаются две кнопки управления историей изменений:

- кнопка ↶ выполняет отмену последнего действия;
- кнопка ↷ выполняет повтор ранее отмененного действия.

Обе команды дублируют привычные клавиатурные сочетания Ctrl+Z и Ctrl+Shift+Z. Если история прошлых состояний пуста, кнопка отмены визуально приглушается и становится неактивной. Аналогично кнопка повтора становится недоступной, если пользователь еще не выполнял отмену или если после отмены уже было совершено новое действие. Такое поведение соответствует общепринятой логике графических редакторов и делает интерфейс предсказуемым.

Следующий блок панели отвечает за выбор типа новой фигуры. В системе предусмотрено пять кнопок добавления объектов:

- □ - размещение прямоугольника;
- ○ - размещение круга или эллиптического элемента;
- ◇ - размещение ромба;
- ≡ - размещение списка;
- ▤ - размещение таблицы.

Кнопки фигур переводят приложение в режим ожидания размещения, а не создают объект сразу. После выбора инструмента пользователь щелкает по холсту, и фигура появляется в указанной точке сетки. Это решение снижает вероятность случайного добавления объекта и делает одинаковым поведение прямоугольника, круга, ромба, списка и таблицы.

Если пользователь передумал, режим размещения можно отменить повторным нажатием кнопки или клавишей Esc. Во время активного режима рядом с панелью появляется подсказка, поэтому пользователь видит, что следующий щелчок по холсту будет не выделением, а установкой нового объекта.

Кнопка со стрелкой включает connect-режим. В нем пользователь сначала выбирает исходную фигуру, затем целевую. Такой режим отделяет создание связи от обычного перемещения объектов и делает построение стрелок более предсказуемым.

Кнопка удаления × delete отображается только тогда, когда на холсте действительно выбран объект или связь. Это решение уменьшает визуальный шум и одновременно подчеркивает контекстность операции: удалять можно только то, что сейчас находится в фокусе. Команда удаления дублируется клавишами Delete и Backspace, если пользователь не редактирует текст в поле ввода.

Кнопка AI открывает интеллектуальную панель. Когда панель уже показана, кнопка визуально выделяется. Это помогает не потеряться в интерфейсе, особенно когда пользователь чередует ручные операции и текстовые команды.

Рядом с AI-кнопкой находятся две команды обмена файлами:

- ↑ .drawio - импорт диаграммы из внешнего файла;
- ↓ .drawio - экспорт текущей диаграммы в совместимый формат.

Их графическая форма также выбрана не случайно: стрелка вверх ассоциируется с загрузкой данных в приложение, а стрелка вниз - с сохранением результата во внешний файл.

Названия содержат расширение .drawio, чтобы пользователь сразу понимал, с каким именно форматом выполняется операция.

Структура панели идет от базовых редакторских команд к расширенным возможностям. Сначала пользователь видит undo/redo, затем инструменты построения объектов, затем связи, тему, AI и обмен файлами. Такой порядок соответствует обычному рабочему циклу построения диаграммы.

Переключатель темы расположен рядом с AI и файловыми командами. На кнопке меняется символ солнца или луны, а выбранный режим сохраняется в браузере. Светлая тема удобна для печатной демонстрации и яркой аудитории, темная - для длительной работы и показа на проекторе.

4.2.3 Рабочая область и поведение холста

Основная рабочая зона реализована как масштабируемый SVG-холст. На фоне отображается сетка с малыми и крупными делениями. Она помогает выравнивать объекты и визуально контролировать расстояние между блоками, что особенно полезно при построении блок-схем и архитектурных диаграмм.

Когда холст пуст, в центре показывается подсказка о двух вариантах начала работы: добавить фигуры вручную или воспользоваться AI-генерацией. Такое пустое состояние не оставляет пользователя перед безмолвным экраном и сразу подсказывает первый шаг.

При выбранном инструменте добавления курсор меняется на перекрестие. Щелчок по холсту переводится в координаты сетки, после чего создается объект с допустимыми размерами и положением. Это обеспечивает более аккуратное расположение элементов уже с момента их появления.

Рабочая область поддерживает панорамирование и масштабирование. Панорамирование выполняется правой кнопкой мыши: пользователь зажимает ее и перемещает изображение холста. Масштабирование осуществляется двумя способами:

- через колесо мыши при удержании Ctrl;
- через клавиатурные сочетания Ctrl+Plus, Ctrl+Minus, а также Ctrl+0 для возврата к исходному масштабу и начальному смещению.

Масштабирование позволяет переключаться между обзором всей схемы и точечной правкой. На маленьком масштабе удобнее оценивать композицию, а при увеличении проще редактировать подписи, ячейки таблицы и расположение связей.

Множественное выделение выполняется прямоугольной рамкой. Пользователь тянет рамку по пустой области холста, и все фигуры, попавшие в нее, выделяются как группа. Это ускоряет перестройку диаграммы, когда нужно сдвинуть сразу несколько блоков.

4.2.4 Выделение, перемещение, изменение размеров и поворот объектов

Каждый объект на холсте может быть выделен щелчком мыши. Выделение сопровождается появлением подсвеченного пунктирного контура вокруг фигуры. Этот контур играет сразу несколько ролей. Во-первых, он визуальное показывает, какой элемент находится в фокусе. Во-вторых, он отделяет сам объект от его управляющих элементов. В-третьих, он служит индикатором готовности к операциям изменения размеров, поворота или удаления.

Перемещение фигуры выполняется простым перетаскиванием. Пользователь нажимает левую кнопку мыши на объекте и переносит его в новую позицию. При этом координаты также привязываются к сетке, что обеспечивает более аккуратное выравнивание схемы. Если выделена группа объектов, перетаскивание любого из них приводит к синхронному перемещению всей группы.

После выделения фигуры становятся доступны маркеры изменения размеров. Пользователь может потянуть такой маркер в нужном направлении и тем самым увеличить или уменьшить объект. Для прямоугольников, ромбов и окружностей изменение размеров действует естественным образом по ширине и высоте. Для таблиц доступно полноценное изменение габаритов, а для списков ширина изменяется более активно, тогда как высота в основном определяется числом строк и автоматически пересчитывается под содержимое.

Наряду с `resize`- маркерами система отображает отдельный маркер поворота. При его использовании вычисляется угол относительно центра объекта, после чего поворот сохраняется как числовой параметр фигуры. Это позволяет не только добиться более выразительного визуального эффекта, но и моделировать диаграммы, в которых некоторые элементы должны быть ориентированы нестандартным образом.

С точки зрения эргономики важно, что все перечисленные действия выполняются прямо на холсте и не требуют перехода к отдельным панелям свойств. Пользователь редактирует объект непосредственно в его визуальном контексте, а не через абстрактные поля формы. Такой подход соответствует логике современных `diagram editors` и особенно полезен в учебной среде, где необходимо быстро экспериментировать с компоновкой.

4.2.5 Редактирование простых фигур

Простые фигуры в приложении представлены прямоугольником, кругом и ромбом. Каждая из них ориентирована на определенный семантический тип содержимого. Прямоугольник удобно использовать для операций, модулей, процессов и отдельных программных компонентов. Круг естественно подходит для обозначения начального и конечного состояния, а также для опорных узлов. Ромб применяется для логических условий, разветвлений и точек принятия решения.

Редактирование текстовой подписи таких фигур организовано через двойной щелчок. После него внутри фигуры появляется встроенное поле ввода, визуальное оформление так, чтобы пользователь по-прежнему воспринимал себя работающим с объектом на схеме, а не с отдельным диалоговым окном. Ввод можно завершить клавишей Enter, потерей фокуса или отменить клавишей Esc.

Преимущество такого решения заключается в его естественности. Пользователь видит фигуру, дважды щелкает по ее содержимому и сразу начинает править надпись. Это особенно удобно в блок-схемах и учебных моделях, где текстовые подписи приходится менять довольно часто. В интерфейсе нет лишних промежуточных состояний, а редактирование происходит непосредственно на том месте, где текст будет отображаться после сохранения.

4.2.6 Работа со списком как отдельным типом объекта

Список в Vision Diagram представляет собой не просто прямоугольник с текстом, а специальный составной объект со структурой заголовка и набора строк. Верхняя часть списка отведена под название, а ниже располагаются отдельные элементы перечня. За счет этого список удобно использовать для перечисления сущностей, модулей, требований, ролей или этапов процесса.

Заголовок списка можно редактировать двойным щелчком аналогично обычной фигуре. Каждая строка списка также поддерживает редактирование по двойному щелчку. Таким образом, пользователь может отдельно менять имя всего блока и содержимое каждого пункта. Это делает список ближе к легковесной структурированной заметке, встроенной в диаграмму.

При выделении списка справа у каждой строки отображается символ удаления X, а в нижней части блока появляется команда + add item. Благодаря этому список можно не только редактировать по содержанию, но и изменять по структуре. Добавление нового пункта

автоматически перестраивает высоту объекта, чтобы все строки оставались видимыми. Удаление строки, в свою очередь, уменьшает объем списка и также влияет на его геометрию.

Подобный подход полезен для инженерных и учебных схем, где часть информации должна быть представлена не в виде таблицы, а именно в виде вертикального перечня. Например, список удобно использовать для набора модулей системы, шагов алгоритма высокого уровня или перечня требований к компоненту.

4.2.7 Работа с таблицей и встроенные управляющие элементы

Табличный объект является одним из наиболее функционально насыщенных элементов интерфейса. Он ориентирован на ситуации, когда внутри диаграммы требуется представить структурированные данные в форме строк и столбцов. Таблица может использоваться для хранения параметров, соответствий, атрибутов сущностей или сравнительных характеристик.

Редактирование таблицы организовано на уровне конкретной ячейки. Пользователь выполняет двойной щелчок по нужной ячейке, после чего в ней появляется поле ввода. Такой подход позволяет воспринимать таблицу как полноценный интерактивный объект, а не как статичную вставку. Важно и то, что первая строка визуально выделяется как заголовочная, что повышает читаемость табличного представления.

При выделении таблицы рядом с ней появляются дополнительные управляющие элементы:

- кнопка + для добавления строки;
- кнопка – для удаления строки;
- кнопка + у нижней границы для добавления столбца;
- кнопка – у нижней границы для удаления столбца.

Эти кнопки имеют контекстный характер: они видимы только в тот момент, когда пользователь действительно работает с таблицей. Такое решение не перегружает интерфейс в обычном режиме и одновременно предоставляет быстрый доступ к структурным операциям. При добавлении строки или столбца система автоматически обновляет внутреннее представление таблицы и при необходимости корректирует ее размеры.

С точки зрения дипломного проекта таблица особенно интересна тем, что она выходит за рамки обычных geometric shapes. В интерфейсе программа демонстрирует способность работать не только с абстрактными блоками и стрелками, но и с более сложными структурированными объектами, в которых пользователь управляет вложенными данными.

4.2.8 Создание связей и настройка типа стрелки

Режим построения связей является важнейшей частью редактора диаграмм, поскольку именно связи превращают набор разрозненных фигур в осмысленную схему. После активации кнопки → приложение переводится в connect- режим. В этом состоянии пользователь должен сначала выбрать исходную фигуру, а затем целевую.

На правой границе каждой фигуры, списка или таблицы появляется специальный круглый порт соединения. Если пользователь выбирает первый объект, система запоминает его как источник связи и выводит подсказку о необходимости щелкнуть по порту целевой фигуры. После выбора второго объекта между ними строится связь, а режим соединения завершается автоматически.

Такой механизм имеет несколько преимуществ. Он снижает вероятность случайного соединения не тех элементов, делает логику действия пошаговой и одновременно хорошо визуализирует сам факт работы именно со связями, а не с перемещением объектов. Кроме того, наличие порта подчеркивает, что соединение создается в определенной точке объекта, а не в любой случайной области.

После выделения уже созданной связи в нижней части интерфейса появляется компактная панель настройки стрелки. В ней пользователь может выбрать один из нескольких вариантов отображения:

- line - простая линия без наконечника;
- filled - стандартная заполненная стрелка;
- empty - пустая стрелка;
- source - стрелка, направленная от источника в обратную сторону;
- both - двусторонняя стрелка.

Наличие такой панели особенно важно для технических и учебных схем, где тип стрелки нередко несет смысловую нагрузку. Например, в одних случаях требуется показать направленный поток управления, в других - двустороннюю связь, а в третьих - просто логическую ассоциацию без выраженного направления.

4.2.9 Строка состояния как средство обратной связи

Нижняя строка состояния выполняет роль постоянного информационного помощника пользователя. В ее левой части отображается количество фигур и связей на текущей диаграмме.

Это полезно не только для общей ориентации, но и для демонстрации того, что приложение поддерживает внутреннюю модель объектов, а не просто рисует графику на экране.

Центральная часть строки состояния посвящена информации о выделении. Если выбрана одна фигура, пользователь видит ее тип, координаты размещения и размеры. Если выбрана группа фигур, строка сообщает число выделенных объектов и подсказывает, что их можно совместно перемещать или удалить. Если выделена связь, строка показывает текущий тип стрелки и напоминает о возможности изменить его. Когда ничего не выбрано, отображается нейтральное сообщение по selection.

Отдельный блок строки состояния показывает активный режим работы. Если пользователь находится в режиме размещения фигуры, строка явно пишет, какой именно объект будет установлен на холст и как отменить этот режим. Если активен режим соединения, отображается подсказка, ожидается ли сейчас выбор исходной фигуры или целевого объекта. Такое решение особенно ценно при насыщенном взаимодействии, когда пользователь может быстро переключаться между несколькими командами и должен безошибочно понимать текущее состояние интерфейса.

В правой части строки состояния находится текущий масштаб холста в процентах. Если масштаб отличается от стандартного значения, он визуально выделяется сильнее. Благодаря этому пользователь сразу замечает, что работает либо в увеличенном, либо в уменьшенном представлении схемы. С точки зрения эргономики это предотвращает ситуацию, когда неточное восприятие размера объектов связано не с их геометрией, а с измененным zoom- уровнем.

4.3 Работа с интеллектуальным помощником

AI-помощник открывается кнопкой AI на верхней панели. Если холст пустой, пользователь может описать будущую схему свободным текстом: например, попросить создать блок-схему алгоритма, структуру модулей или простую схему процесса. В этом случае первый ответ обычно создает основу диаграммы.

Если на холсте уже есть объекты, клиент передает их серверу как контекст. Тогда пользователь может не только создать новую диаграмму, но и попросить добавить блок, удалить связь, переименовать элемент или проанализировать текущее содержимое рабочей области.

Результат запроса выражается либо в изменении диаграммы, либо в сообщении в истории диалога. Если результат оказался неудачным, пользователь может применить undo или исправить схему вручную, поэтому AI не лишает его контроля над итоговой моделью.

При работе с помощником лучше использовать конкретные запросы. Например, команда «построй блок-схему обработки заказа с этапами создание, проверка, оплата и завершение» дает модели больше структуры, чем короткая фраза «сделай схему». Чем яснее перечислены элементы, тем надежнее получается результат.

Для изменения готовой диаграммы полезно явно указывать действие: «добавь», «удали», «переименуй», «проанализируй». Серверная часть все равно сводит ответ модели к ограниченному набору операций, поэтому короткая и точная команда обычно работает лучше длинного описания с несколькими неоднозначными условиями.

После ответа модели рекомендуется посмотреть на холст и убедиться, что структура соответствует задаче. Это нормальная часть работы с генеративной системой: AI ускоряет получение заготовки, а окончательную проверку логики и оформления выполняет пользователь.

4.3.1 Структура и элементы панели интеллектуального помощника

Панель AI-помощника закреплена справа снизу и занимает ограниченную область экрана. Она не перекрывает весь холст, поэтому пользователь видит и диалог, и диаграмму. Такой формат похож на рабочий чат рядом с редактором, а не на отдельный режим приложения.

В заголовке панели указано AI ASSISTANT и модель llama3.1. Справа находится кнопка закрытия. Это простые элементы, но они важны для восприятия: пользователь понимает, какая часть интерфейса отвечает за интеллектуальный режим, и может скрыть ее, когда работает только вручную.

Под заголовком расположен индикатор контекста. Если на холсте есть объекты, он показывает их количество и сообщает, что AI может работать с текущей диаграммой. Если холст пуст, это также отображается явно. Благодаря этому пользователь понимает, какой сценарий будет выполнен: создание новой схемы или изменение существующей.

История сообщений оформлена как диалог. Сообщения пользователя и ответы системы различаются цветом и выравниванием, поэтому легко проследить ход работы: какая команда была отправлена и какой результат вернул помощник.

Поле ввода меняет подсказку в зависимости от состояния холста. Для пустой диаграммы оно предлагает описать будущую схему, а для заполненной - добавить, изменить, удалить или проанализировать объекты. Это небольшая, но полезная деталь: интерфейс сам подсказывает допустимые действия.

4.3.2 Логика взаимодействия с AI и типы пользовательских запросов

Первый сценарий работы с AI - создание новой диаграммы. Пользователь описывает процесс или структуру, сервер возвращает action replace, и клиент заменяет текущее состояние на набор фигур и связей, полученный от модели.

Второй сценарий - добавление элементов. Если диаграмма уже есть, пользователь может попросить вставить новый модуль, проверку, список или таблицу. Сервер возвращает add, а клиент присоединяет новые объекты к существующей работе.

Третий сценарий - изменение уже созданных объектов. В этом случае ответ update содержит только те поля, которые нужно поменять: текст, размеры, координаты или другие параметры. Такой режим особенно важен, потому что он показывает работу с редактируемой моделью, а не с одноразовой картинкой.

Четвертый сценарий связан с удалением объектов и связей, пятый - с анализом. При remove клиент удаляет указанные идентификаторы, а при analyze диаграмма остается без изменений, но пользователь получает текстовое описание ее содержимого.

Панель также распознает команды отмены и повтора. Сообщения вроде «отмени», «верни назад», undo, «повтори» или redo вызывают локальные методы истории изменений. Это показывает, что AI-панель связана не только с сервером, но и с обычными инструментами редактора.

4.3.3 Практическая роль AI-панели в пользовательском процессе

В пользовательском процессе AI-панель играет роль ускорителя. Она помогает быстро получить заготовку, внести типовое изменение или понять содержимое холста, но окончательное редактирование остается за пользователем.

Один типичный сценарий начинается с пустого холста: пользователь описывает процесс, получает первичную схему, затем вручную двигает блоки, меняет стрелки и уточняет подписи. Другой сценарий начинается с ручной схемы, после чего AI добавляет недостающий блок или кратко объясняет уже собранную структуру.

За счет такого сочетания Vision Diagram демонстрирует практическое включение естественного языка в инженерный workflow. AI не является внешней надстройкой: он работает с тем же состоянием, которое редактируется на холсте и сохраняется в модели приложения.

4.4 Импорт и экспорт файлов

Для импорта диаграммы необходимо выбрать команду открытия файла и указать XML-файл формата draw.io. После успешного разбора содержимого схема будет загружена в рабочую область и станет доступной для дальнейшего редактирования.

Для экспорта диаграммы пользователь выбирает команду сохранения. Приложение формирует XML-структуру, совместимую с draw.io, и инициирует загрузку файла в браузере. Такой файл может быть открыт во внешних системах и использован как результат работы редактора.

При использовании импорта следует учитывать, что наилучшие результаты достигаются для тех диаграмм, чья структура близка к поддерживаемым в Vision Diagram типам фигур. Если внешний файл содержит сложные специализированные элементы, не имеющие прямого аналога во внутренней модели приложения, часть оформления может быть интерпретирована приближенно. Однако для основных сценариев учебной и прикладной работы с блоками, таблицами и списками формат обмена оказывается достаточно удобным.

Экспорт, в свою очередь, полезен не только как способ сохранения результата, но и как средство интеграции с другими инструментами. Пользователь может начать работу в Vision Diagram, задействовать интеллектуальный модуль для чернового построения структуры, а затем перенести полученный результат в другой редактор для дополнительной доработки или включения в более крупный документ.

4.4.1 Поведение интерфейса при импорте

С пользовательской точки зрения импорт реализован как максимально простая операция. Нажатие кнопки  на верхней панели не открывает отдельное сложное меню, а вызывает стандартный диалог выбора файла через скрытый элемент ввода. После выбора документа приложение считывает его содержимое и пытается интерпретировать XML-структуру как поддерживаемую диаграмму.

Если разбор файла проходит успешно, диаграмма загружается в рабочую область как новое состояние проекта. Если же файл поврежден или не соответствует ожидаемому формату, система уведомляет пользователя об ошибке. Такой сценарий важен с точки зрения надежности интерфейса: пользователь получает понятный результат и не сталкивается с «молчаливым» отказом.

4.4.2 Поведение интерфейса при экспорте

Команда `↓ .drawio` выполняет обратную задачу: она сериализует текущее состояние рабочей области и инициирует сохранение файла на стороне браузера. Для пользователя экспорт выглядит как единичное действие, однако на практике за ним стоит формирование совместимого XML-представления фигур, таблиц, списков и связей.

Эргономическая ценность этой кнопки в том, что она делает результат работы переносимым. Пользователь воспринимает Vision Diagram не как замкнутую демонстрационную среду, а как инструмент, способный обмениваться результатами с внешними системами. В дипломной работе это усиливает практическую значимость проекта, а в пользовательском интерфейсе - завершенность сценария «создал, отредактировал, сохранил».

4.5 Типовые ошибки и способы их устранения

При эксплуатации системы пользователь может столкнуться с рядом типовых проблем. Большинство из них связано не с ошибками бизнес- логики, а с особенностями окружения и взаимодействия нескольких программных компонентов.

Первая распространенная проблема - отсутствие ответа интеллектуального помощника. Наиболее вероятная причина заключается в том, что не запущен сервер Vision Diagram или локальная модель Ollama недоступна. В этом случае необходимо проверить, что сервер отвечает по маршруту `health`, а команда `ollama list` возвращает список доступных моделей.

Вторая проблема - невозможность корректного импорта файла. Как правило, это означает, что выбранный файл не соответствует ожидаемому XML- формату или содержит структуру, не интерпретируемую текущей версией приложения. В такой ситуации рекомендуется проверить целостность исходного файла и, по возможности, использовать более простой вариант диаграммы.

Третья проблема связана с неожиданным результатом генерации схемы. Поскольку интеллектуальный модуль опирается на вероятностную языковую модель, слишком общий или неоднозначный запрос может привести к нежелательной структуре диаграммы. В этом случае рекомендуется переформулировать запрос более конкретно либо воспользоваться ручным редактированием и историей изменений.

Четвертая проблема может проявляться в несовместимости версий среды выполнения. Например, часть инструментов сборки фронтенда чувствительна к используемой версии Node.js.

В этом случае пользователю необходимо обновить среду исполнения до совместимой версии и повторить запуск проекта.

4.6 Ограничения текущей версии и рекомендации пользователю

Текущая версия Vision Diagram реализует ядро интеллектуального редактора, однако как дипломный прототип имеет ряд естественных ограничений. Система ориентирована на локальный сценарий использования и не содержит развитого серверного хранилища проектов. Состояние диаграммы сохраняется в браузере, что удобно для демонстраций и индивидуальной работы, но пока не заменяет полноценную систему управления документами.

Интеллектуальный модуль пока работает с ограниченным набором действий и типов фигур. Этого достаточно для демонстрации идеи и учебных сценариев, но для промышленного использования потребуются расширенная библиотека объектов, дополнительные проверки целостности схем и более развитая автокомпоновка.

Пользователю стоит рассматривать Vision Diagram как инструмент быстрого чернового моделирования. Наиболее надежный сценарий - получить заготовку через AI или вручную, затем проверить структуру глазами и доработать ее обычными средствами редактора.

В текущей версии доступны две темы оформления - светлая и темная. Они задаются через централизованные цветовые токены и сохраняются в браузере, но пользователь пока не может создавать собственные палитры или настраивать цвета отдельных типов диаграмм. Это можно рассматривать как отдельное направление развития интерфейса.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы разработано программное средство Vision Diagram - браузерный редактор схем и таблиц с поддержкой интеллектуального помощника. В процессе работы были изучены существующие решения, выбрана клиент-серверная архитектура и реализован прототип, который можно запустить локально.

Практический результат включает интерактивный SVG-холст, панель инструментов, управление состоянием через Zustand, операции undo/redo, соединение фигур, работу со списками и таблицами, импорт и экспорт draw.io, переключение светлой и темной темы, а также серверное взаимодействие с Ollama.

Особенно важным результатом стала формализованная связь между языковой моделью и редактором. Модель возвращает не произвольный текст, а действие над диаграммой. Благодаря этому клиент может проверить ответ, применить его к состоянию и сохранить возможность ручной корректировки.

Поставленная цель достигнута: создано приложение, которое объединяет классический редактор диаграмм и AI-помощника. Оно подходит для учебных задач, демонстрации алгоритмов, описания небольших архитектурных решений и подготовки черновых схем для дальнейшей доработки.

В ходе разработки подтверждено, что даже сравнительно компактное веб-приложение может объединять ручное редактирование, файловый обмен и генерацию структуры по естественному языку. При этом качество результата повышается, если AI используется как помощник, а не как единственный источник окончательной схемы.

Работа имеет и методическую ценность. В ней можно проследить полный путь данных: от пользовательской команды и объекта на холсте до JSON-запроса, серверной обработки, ответа модели и обновления состояния в браузере. Такой разбор делает проект понятным не только как готовую программу, но и как пример архитектуры интеллектуального редактора.

Дополнительно была реализована поддержка светлой и темной темы. Это улучшило эргономику интерфейса: приложение удобнее показывать в разных условиях освещения, а пользователь может выбирать визуальный режим без перезапуска программы.

Перспективы дальнейших исследований

Дальнейшее развитие Vision Diagram связано с двумя направлениями: расширением редакторских возможностей и усилением интеллектуальной части.

К основным направлениям развития можно отнести:

- реализацию полноценного сохранения проектов в отдельные файлы и облачное хранилище;
- добавление экспорта в форматы PNG, PDF и SVG;
- внедрение автоматической компоновки диаграммы на основе алгоритмов layout;
- расширение набора поддерживаемых фигур и нотаций;
- реализацию проверки логической целостности схемы;
- добавление пользовательских аккаунтов и разграничения доступа к проектам;
- поддержку совместного редактирования;
- подключение нескольких моделей искусственного интеллекта и выбор модели из интерфейса;
- формирование текстового описания диаграммы и генерацию документации на основе схемы.

Реализация этих направлений позволит постепенно превратить текущий локальный прототип в более зрелую среду визуального проектирования.

Отдельное перспективное направление - развитие серверной части. В будущем можно добавить хранение проектов, версионирование, журнал изменений, совместную работу и синхронизацию между устройствами. Тогда Vision Diagram сможет использоваться не только как локальный редактор, но и как коллективная среда моделирования.

Еще одно направление связано с интеллектуальным анализом готовых схем. Система может научиться находить несвязанные элементы, замечать нарушенную логику блок-схем, предлагать улучшение компоновки и формировать пояснительный текст на основе содержимого диаграммы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. diagrams.net Documentation [Электронный ресурс]. - URL: <https://www.drawio.com/doc/> (дата обращения: 11.05.2026).
2. draw.io. mxGraphModel and file format overview [Электронный ресурс]. - URL: <https://www.drawio.com/doc/faq/format-diagram-file> (дата обращения: 11.05.2026).
3. React Documentation [Электронный ресурс]. - URL: <https://react.dev/> (дата обращения: 11.05.2026).
4. TypeScript Documentation [Электронный ресурс]. - URL: <https://www.typescriptlang.org/docs/> (дата обращения: 11.05.2026).
5. Vite Guide [Электронный ресурс]. - URL: <https://vite.dev/guide/> (дата обращения: 11.05.2026).
6. Node.js Documentation [Электронный ресурс]. - URL: <https://nodejs.org/docs/latest/api/> (дата обращения: 11.05.2026).
7. Express Documentation [Электронный ресурс]. - URL: <https://expressjs.com/> (дата обращения: 11.05.2026).
8. Zustand Documentation [Электронный ресурс]. - URL: <https://zustand.docs.pmnd.rs/> (дата обращения: 11.05.2026).
9. Ollama Docs [Электронный ресурс]. - URL: <https://ollama.com/> (дата обращения: 11.05.2026).
10. SVG 2 [Электронный ресурс] / W3C. - URL: <https://www.w3.org/TR/SVG2/> (дата обращения: 11.05.2026).
11. ГОСТ 19.101- 77. Единая система программной документации. Виды программ и программных документов.
12. ГОСТ 19.105- 78. Единая система программной документации. Общие требования к программным документам.
13. ГОСТ 19.402- 78. Единая система программной документации. Описание программы.
14. ГОСТ 19.505- 79. Единая система программной документации. Руководство оператора. Требования к содержанию и оформлению.

ПРИЛОЖЕНИЕ А. ОПИСАНИЕ ПРОГРАММЫ

Описание программы подготовлено в соответствии с требованиями ЕСПД к программному документу данного вида [11]–[13].

А.1 Общие сведения

Наименование программы: Vision Diagram.

Тип программного средства: интеллектуальное веб-приложение для создания, редактирования, импорта, экспорта и анализа диаграмм и таблиц.

Область применения: учебные задачи, подготовка технической документации, визуализация алгоритмов, моделирование структуры программных систем и бизнес-процессов.

А.2 Функциональное назначение

Программа предназначена для ускорения перехода от текстового описания предметной области к визуальной модели. Пользователь может строить схему вручную, создавать табличные элементы, соединять объекты связями, выполнять импорт и экспорт диаграмм в совместимом формате, а также использовать интеллектуальный модуль для генерации и модификации диаграммы по текстовому запросу.

Функциональные ограничения текущей версии связаны с локальным сценарием эксплуатации, отсутствием многопользовательской работы и ориентацией на набор фигур, реализованный в исходном коде проекта.

А.3 Описание логической структуры

Программа состоит из двух основных частей:

- клиентской части, выполняемой в браузере и отвечающей за графический интерфейс, хранение локального состояния диаграммы и операции редактирования;
- серверной части, принимающей текстовые запросы, взаимодействующей с Ollama и возвращающей структурированные действия над диаграммой.

Связь между частями организована по HTTP через JSON- структуры. Внутреннее состояние диаграммы представлено совокупностью фигур, связей и параметров рабочей области.

A.4 Используемые технические средства

Для работы программы необходим персональный компьютер с современной операционной системой, установленной средой Node.js, менеджером пакетов npm, браузером, поддерживающим современные web- стандарты, а также локально установленным сервисом Ollama для сценариев интеллектуальной генерации.

A.5 Входные и выходные данные

К входным данным относятся действия пользователя в интерфейсе, текстовые запросы к интеллектуальному модулю, а также XML-файлы диаграмм в формате draw.io, передаваемые при импорте.

Выходными данными являются визуальное состояние рабочей области, экспортируемый XML-файл диаграммы, а также текстовые и структурированные ответы интеллектуального модуля.

ПРИЛОЖЕНИЕ Б. РУКОВОДСТВО ОПЕРАТОРА

Руководство оператора подготовлено в соответствии с требованиями ЕСПД к эксплуатационным документам [11], [12], [14].

Б.1 Назначение документа

Настоящее руководство предназначено для пользователя программы Vision Diagram и содержит сведения, необходимые для запуска, базовой настройки и практической эксплуатации приложения.

Б.2 Условия выполнения программы

Перед началом работы необходимо установить зависимости клиентской и серверной частей проекта, проверить наличие Node.js и npm, а также при необходимости убедиться в доступности локальной модели Ollama. Для интеллектуального режима сервер и Ollama должны быть запущены одновременно.

Б.3 Подготовка к работе

Для подготовки к работе требуется:

- открыть каталог клиентской части проекта и выполнить установку зависимостей;
- открыть каталог серверной части проекта и выполнить установку зависимостей;
- запустить клиентское приложение;
- запустить серверное приложение;
- убедиться, что Ollama доступен и выбранная модель загружена.

Б.4 Порядок работы

После запуска пользователь может создать новую диаграмму, добавляя фигуры через панель инструментов, или сразу воспользоваться интеллектуальным помощником. Для ручного редактирования доступны операции перемещения, изменения размеров, поворота, соединения объектов и редактирования текстового содержимого. Для обмена с внешними средствами используются команды импорта и экспорта формата draw.io.

При использовании интеллектуального помощника рекомендуется формулировать запросы кратко и однозначно. Если требуется сложное изменение диаграммы, целесообразно разбить его на несколько последовательных запросов.

Б.5 Сообщения оператору и нештатные ситуации

Если клиентская часть не открывается, необходимо проверить корректность установки зависимостей и отсутствие ошибок сборки. Если интеллектуальный модуль не отвечает, следует проверить запуск серверной части и доступность Ollama. При проблемах с импортом рекомендуется убедиться в корректности исходного XML-файла. При необходимости пользователь может восстановить предыдущее состояние диаграммы через механизм отмены действий.

ПРИЛОЖЕНИЕ В. ПРОТОКОЛ ФУНКЦИОНАЛЬНОГО ТЕСТИРОВАНИЯ

В.1 Цель испытаний

Целью функционального тестирования являлась проверка работоспособности основных пользовательских сценариев программы Vision Diagram, а также подтверждение корректного взаимодействия клиентской и серверной частей при ручном и интеллектуальном режимах работы.

В.2 Состав испытательной среды

Испытательная среда включала:

- локальный персональный компьютер под управлением настольной операционной системы;
- установленную среду Node.js и менеджер пакетов npm;
- клиентскую часть приложения, запускаемую средствами Vite;
- серверную часть, запускаемую на платформе Node.js и Express;
- локальный сервис Ollama с доступной языковой моделью;
- браузер, поддерживающий современные web- стандарты.

В.3 Перечень контрольных сценариев

Функциональное тестирование выполнялось по следующим контрольным сценариям:

- запуск клиентской части и открытие рабочей области в браузере;
- запуск серверной части и проверка служебного маршрута health;
- создание прямоугольника, ромба, эллипса и других базовых фигур на холсте;
- выделение фигуры, изменение ее размеров и поворот;
- редактирование текстового содержимого фигуры;
- создание табличного объекта, изменение структуры таблицы и редактирование ячеек;
- соединение двух фигур связью и проверка автоматического перестроения маршрута;
- отмена и повтор действий через историю изменений;
- генерация новой диаграммы по текстовому описанию;
- изменение существующей диаграммы по контекстному запросу;
- импорт диаграммы из файла draw.io;
- экспорт созданной диаграммы в совместимый XML- формат.

В.4 Ожидаемые результаты

Для каждого сценария проверялось достижение конкретного ожидаемого результата.

При запуске клиентской части ожидаемым результатом являлось открытие редактора без ошибок сборки и отображение основных элементов интерфейса. При запуске серверной части ожидалось успешное получение ответа маршрута health и доступность обработки запросов от клиента.

Для операций ручного редактирования ожидаемым результатом являлось корректное отображение фигур, отсутствие потери состояния при изменении параметров объектов, а также правильная фиксация действий в истории изменений. Для таблиц дополнительно проверялась сохранность структуры строк, столбцов и значений ячеек.

Для интеллектуального режима ожидалось, что сервер принимает текстовый запрос, взаимодействует с Ollama, возвращает структурированный ответ, а клиентская часть корректно интерпретирует его как действие над диаграммой. В случае неоднозначного результата проверялась устойчивость программы к частично неполным данным.

Для импорта и экспорта ожидалась корректная сериализация и десериализация основных типов фигур и связей без критической потери структуры диаграммы.

В.5 Итоги испытаний

Проведенные испытания показали, что ключевые пользовательские сценарии программы выполняются корректно. Наиболее чувствительным участком остается интеллектуальный режим, так как он зависит от формулировки запроса и особенностей ответа языковой модели. Тем не менее применяемая нормализация ответа и возможность ручной доработки результата позволяют считать систему пригодной для учебной и демонстрационной эксплуатации.

ПРИЛОЖЕНИЕ Г. СПЕЦИФИКАЦИЯ СТРУКТУР ДАННЫХ И ОБМЕНА

Г.1 Общие положения

Внутренний обмен данными в программе Vision Diagram построен на формализованных структурах, описывающих фигуры, связи и действия интеллектуального модуля. Такой подход позволяет не только хранить и редактировать состояние диаграммы, но и выполнять контролируемое интеллектуальное преобразование без прямой генерации графики.

Г.2 Структура фигуры

Каждая фигура описывается набором обязательных и дополнительных полей.

К обязательным полям относятся:

- уникальный идентификатор фигуры;
- тип фигуры;
- координаты размещения на рабочей области;
- размеры объекта;
- текстовое содержимое;
- визуальные параметры, необходимые для корректного отображения.

Дополнительные поля зависят от типа объекта. Для таблиц это структура строк, столбцов и ячеек. Для списков - набор элементов и параметры их отображения. Для фигур с поворотом - угол вращения. Подобная схема позволяет хранить в единой модели как простые геометрические элементы, так и составные объекты.

Г.3 Структура связи

Связь содержит идентификатор, ссылку на исходную фигуру, ссылку на целевую фигуру и тип стрелки. На клиентской стороне графическое представление связи вычисляется динамически по координатам соединяемых объектов. Это упрощает перестроение диаграммы при перемещении фигур и уменьшает объем данных, которые нужно хранить в постоянном состоянии.

Г.4 Структура действия интеллектуального модуля

Ответ сервера после обращения к языковой модели интерпретируется как действие над диаграммой. В текущей версии поддерживаются следующие типы действий:

- replace - полная замена содержимого диаграммы;

- add - добавление новых фигур и связей;
- update - изменение параметров существующих объектов;
- remove - удаление выбранных фигур и связей;
- analyze - текстовое описание текущей диаграммы без изменения ее состояния.

Для каждого действия сервер возвращает только тот набор данных, который необходим клиенту для применения результата. Благодаря этому обмен остается компактным и предсказуемым.

Г.5 Пример логики обмена

При создании новой диаграммы клиент формирует текстовый запрос пользователя и передает его серверу. Сервер дополняет запрос системной инструкцией, содержащей перечень допустимых типов фигур, границы рабочей области и требования к JSON-ответу. После получения результата от Оллата выполняется нормализация: проверяются названия типов, корректируются размеры, координаты и состав полей. Затем клиент применяет результат к локальному состоянию диаграммы.

Если пользователь изменяет уже существующую схему, в запрос дополнительно включается текущее состояние рабочей области. Это позволяет интеллектуальному модулю выполнять контекстное редактирование, а не только генерацию с нуля.

Г.6 Значение спецификации для сопровождения

Выделение явной спецификации структур данных важно не только для реализации текущей версии проекта, но и для его дальнейшего развития. Наличие формализованной модели облегчает отладку, расширение библиотеки фигур, внедрение новых действий интеллектуального модуля и подключение альтернативных моделей. Кроме того, такая спецификация упрощает подготовку пользовательской и программной документации, поскольку все ключевые сущности системы описываются единообразно и прозрачно.